Analytical Skills for Business (WS 2025)

Business Administration (M. A.)

© Benjamin Gross

29.05.2026 07:28

This document holds the course material for the Analytical Skills for Business course in the Master of Business Administration program. It discusses version control systems such as Git and GitHub for efficient team collaboration, offers an overview of no-code and low-code tools for data analytics including Tableau, Power BI, QlikView, makeML, PyCaret, RapidMiner, and KNIME, and introduces key programming languages such as R, Python, and SQL alongside essential programming concepts like syntax, libraries, variables, functions, objects, conditions, and loops. In addition, it covers working with modern development environments, including Unix-like systems, containers, APIs, Jupyter, and RStudio, and sets expectations for project submissions and evaluation.

Table of contents

1 Introduction

Computer science is the study of computers and computation, spanning theoretical and algorithmic foundations, the design of hardware and software, and practical uses of computing to process information. It encompasses core areas such as

- algorithms and data structures
- computer architecture
- programming languages and software engineering
- databases and information systems
- networking and communications
- graphics and visualization
- human-computer interaction
- intelligent systems.

The field draws on mathematics and engineering—using concepts like

- binary representation
- Boolean logic
- complexity analysis

to reason about what can be computed and how efficiently.

Emerging in the 1960s as a distinct discipline, computer science now sits alongside computer engineering, information systems, information technology, and software engineering within the broader computing family. Its reach is inherently interdisciplinary, intersecting with domains from the natural sciences to business and the social sciences. Beyond technical advances, the discipline engages with societal and professional issues, including

- reliability
- security
- privacy
- intellectual property

in a networked world (Britannica, 2025).

1.1 Implementing version control systems

Version control systems are essential tools for managing code, tracking changes, and facilitating collaborative development in modern development projects (Çetinkaya-Rundel & Hardin, 2021). These systems enable teams to work efficiently on shared codebases while maintaining a complete history of all modifications, ensuring reproducibility and accountability in data analysis workflows.

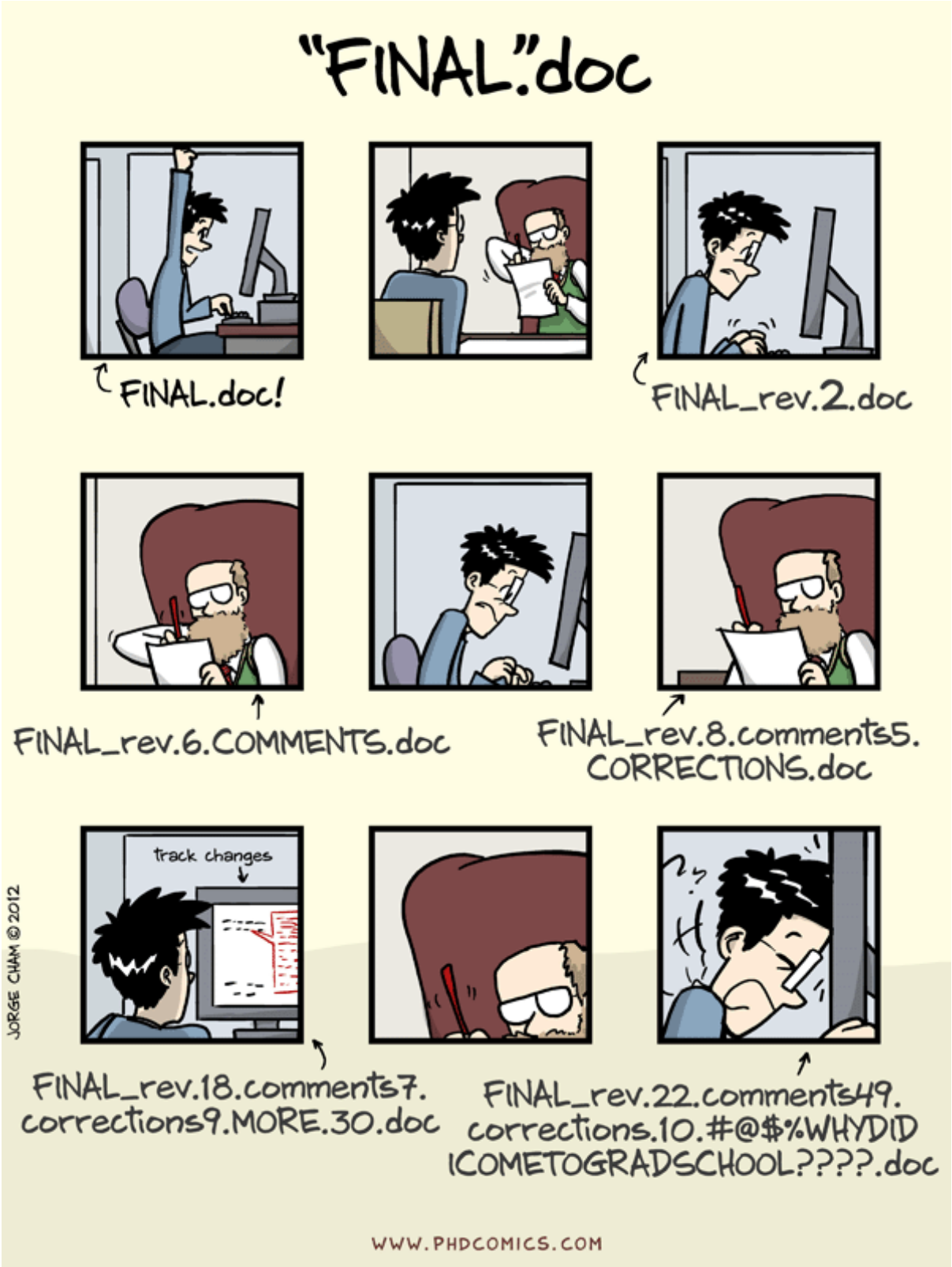
1.1.1 Core Concepts

Version control systems provide systematic approaches to managing changes in documents, programs, and other collections of information:

- **Repository**: A central storage location containing all project files and their complete revision history

- **Commit**: A snapshot of the project at a specific point in time, representing a set of changes
- **Branch**: An independent line of development allowing parallel work on different features
- **Merge**: The process of integrating changes from different branches back together

Figure 1: **Source:** *https://uidaholib.github.io/get-git/1why.html*



1.1.2 Git: Distributed Version Control

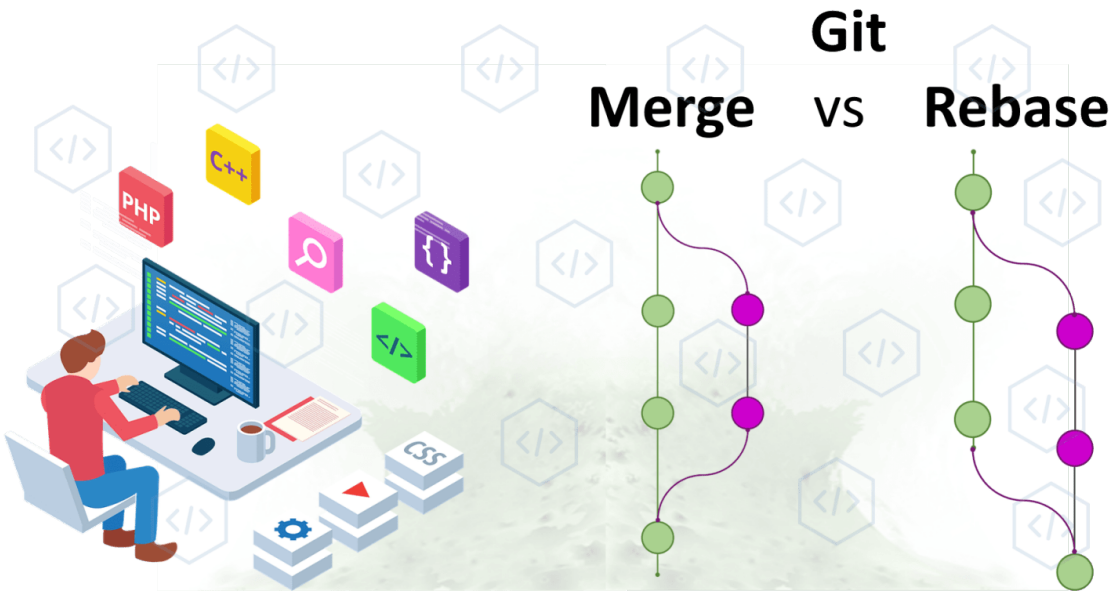
Git is a distributed version control system that tracks changes in files and coordinates work among multiple contributors. It was created by **Linus Torvalds** (creator of Linux) in 2005 and has since become the de facto standard for version control in software development. Key characteristics include:

Local Repository: Each user maintains a complete copy of the project history, enabling offline work and faster operations.

Staging Area: An intermediate area where changes are prepared before being committed to the repository.

Branching and Merging: Lightweight branching allows for experimental development without affecting the main codebase. Merging integrates changes from different branches. In Open Source projects often Pull Requests are used to propose and discuss changes before merging. In the coporate world often Merge Requests are used. There is a difference between merging and rebasing. Merging creates a new commit that combines the histories of two branches, while rebasing rewrites the commit history to create a linear sequence of changes as you see on the figure below.

Figure 2: **Source:** *https://i0.wp.com/digitalvarys.com/wp-content/uploads/2020/03/Git-Merge-and-Rebase.png?resize=1536%2C843&ssl=1*



Distributed Workflow: No single point of failure, as every user has a complete backup of the project.

1.1.3 GitHub: Cloud-Based Collaboration Platform

GitHub is a web-based hosting service for Git repositories that adds collaboration features and project management tools:

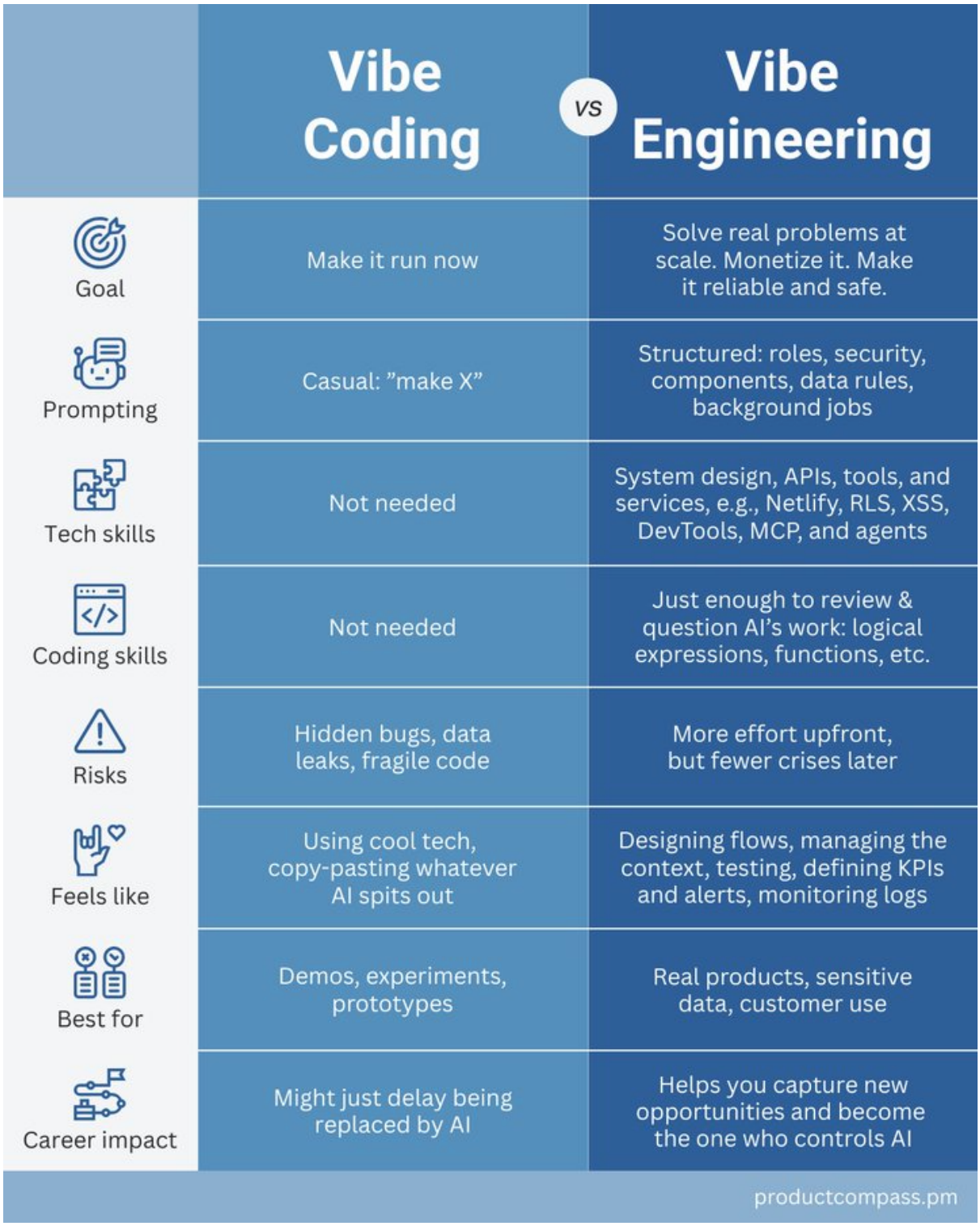
- **Remote Repositories**: Centralized storage accessible from anywhere with internet connectivity.
- **Pull Requests**: Structured code review process for integrating changes.
- **Issue Tracking**: Built-in project management for tracking bugs and feature requests.
- **Actions and CI/CD**: Automated workflows for testing and deployment.
- **Documentation**: Integrated wiki and README support for project documentation.

The combination of Git and GitHub creates a powerful ecosystem for collaborative analytics projects, ensuring code quality, facilitating peer review, and maintaining comprehensive project documentation (G. GeeksforGeeks, 2024).

See also Collaborating with Git and GitHub by Prof. Dr. Huber about using git and Github for collaboration.

For students Github offers a free educational plan with additional features! Included you will find access to Github Copilot, a AI based code completion tools like. These are powerful gadgets to support your coding activities, but not a replacement for learning programming and coding by yourself as you can see on this image:

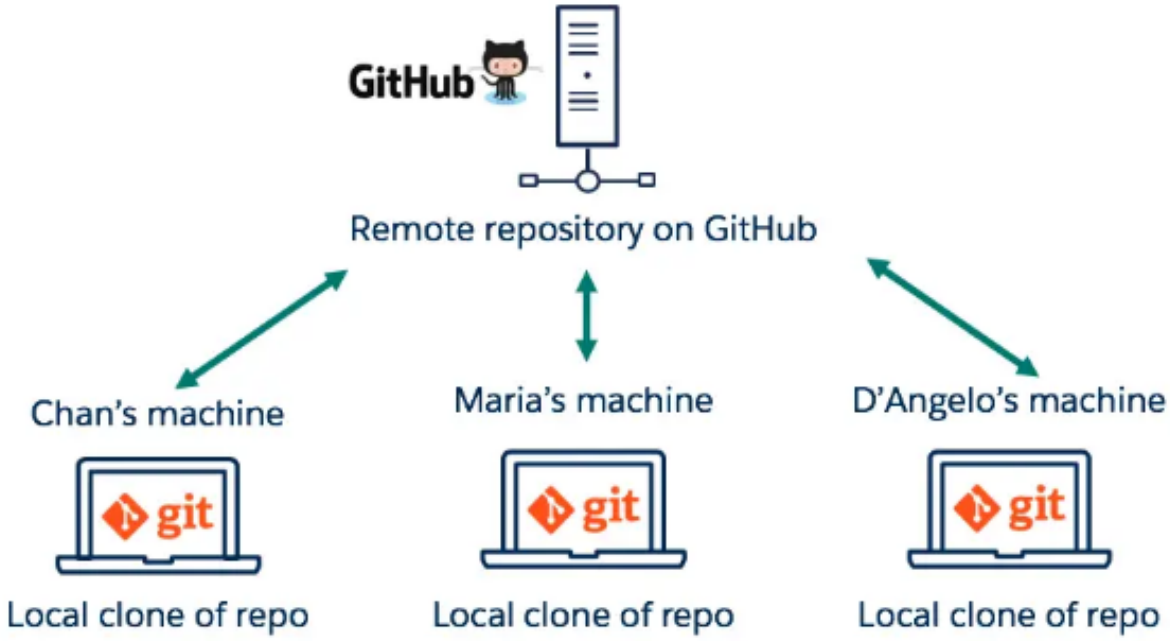
Figure 3: **Source:** *LinkedIn*



1.1.4 Comparison of Git and GitHub

On this image you can see the integration of Git in GitHub:

Figure 4: **Source:** *https://marce10.github.io/ciencia_reproducible/intro_a_git_y_github.html*



1.1.5 Business Analytics Applications

In business analytics contexts, version control systems provide:

- **Reproducible Analysis**: Complete tracking of analytical scripts and data processing steps
- **Collaborative Research**: Multiple analysts can work simultaneously on different aspects of projects
- **Model Versioning**: Systematic management of machine learning models and their evolution
- **Data Governance**: Audit trails for compliance and regulatory requirements
- **Backup and Recovery**: Protection against data loss and accidental modifications

1.1.6 A lot of documentation available

For more coverage of version control concepts, implementation strategies, and best practices, see:

- **Theoretical Foundation**: Introduction to Modern Statistics provides context on reproducible research practices and collaborative analytics (Çetinkaya-Rundel & Hardin, 2021)
- **Git Resources**: Git Cheat Sheet offers quick reference for common Git commands
- **GitHub Documentation**: GitHub Manual contains detailed guidance on platform features
- **Online Resources**:
  - GeeksforGeeks Git vs GitHub Guide provides practical comparisons and use cases (G. GeeksforGeeks, 2024)
  - Official GitHub Documentation offers authoritative guidance on getting started (GitHub, 2024)

Understanding version control systems is fundamental for modern business analytics, enabling collaborative development, ensuring reproducibility, and maintaining professional standards in data science projects.

1.2 Overview on Programming languages

- **R:** R is a programming language and free software environment used for statistical computing and graphics supported by the R Foundation for Statistical Computing. It is widely used among statisticians and data miners for developing statistical software and data analysis.
- **Python:** Python is a versatile programming language widely used in data science and analytics. It has a rich ecosystem of libraries such as Pandas, NumPy, and Matplotlib that facilitate data manipulation, analysis, and visualization. It can also be used to develop software as it is a general-purpose programming language which utilizes an object-oriented programming paradigm.
- **SQL:** SQL (Structured Query Language) is the standard language for managing and querying relational databases. It is essential for data extraction, transformation, and loading (ETL) or extraction, loading and transformation (ELT) processes in analytics workflows. Modern databases like PostgreSQL, MySQL, and SQLite use SQL for data manipulation and retrieval and just have slightly different dialects.

1.3 Elements of programming languages

- **Syntax:** the set of rules that defines the combinations of symbols that are considered to be correctly structured programs in that language.
- **Libraries:** collections of pre-written code that users can call upon to save time and effort.
- **Variables:** named storage locations in a program that hold values.
- **Functions:** reusable blocks of code that perform a specific task.
- **Objects:** instances of classes that encapsulate data and behavior.
- **Conditions:** statements that control the flow of execution based on certain criteria.
- **Loops:** constructs that repeat a block of code multiple times.

1.4 Development environments

- **Unix-like systems:**Most popular Unix like system is Linux. It is understood as a family of open source Unix-like operating systems based on the Linux kernel and mostly used in form of distributions such as Ubuntu, Debian, Fedora, CentOS and Alpine Linux. The latter is known for its simplicity and efficiency and is often used as the base image for Docker containers (container host os).
- **Containers:** Docker is a production ready containerization service. On hub.docker.com you can store images that could be pulled. It acts as the main public registry for Docker images. Container are a standard unit of software that packages up code and all its dependencies so the application runs quickly and reliably from one computing environment to another. A Docker container image is a lightweight, standalone, executable package of software that includes everything needed to run an application: code, runtime, system tools, system libraries and settings.
- **APIs:** (Application Programming Interface): An API is a set of definitions and protocols for building and integrating application software. It allows different software systems to communicate with each other. APIs are used to enable the integration of different systems,

allowing them to share data and functionality. Examples include RESTful APIs, SOAP APIs, and GraphQL APIs.

- **Jupyter:** Jupyter is an open-source web application that allows you to create and share documents that contain live code, equations, visualizations, and narrative text. It supports various programming languages, including Python, R, and Julia. Jupyter is widely used for data analysis, machine learning, and scientific computing.
- **IDEs:** (Integrated Development Environments): IDEs are software applications that provide comprehensive facilities to computer programmers for software development. They typically include a code editor, a debugger, and build automation tools. Examples of popular IDEs include:
  - **RStudio:** RStudio is an integrated development environment (IDE) for R, a programming language for statistical computing and graphics. RStudio provides a user-friendly interface for writing and debugging R code, as well as tools for data visualization and reporting.
    1. Install R from CRAN
    2. Install RStudio from RStudio
    3. Install git (if not already installed) from git-scm
    4. Follow the lecture to clone the Course GitHub repository and open the project in RStudio https://github.com/DrBenjamin/Analytical-Skills-for-Business
  - **Visual Studio Code (VS Code):** VS Code is a free source-code editor made by Microsoft for Windows, Linux and macOS. It includes support for debugging, embedded Git control, syntax highlighting, intelligent code completion, snippets, and code refactoring. It is highly customizable, allowing users to change the theme, keyboard shortcuts, preferences, and install extensions that add additional functionality.

1.5 Overview on no-code and low-code tools for data analytics

- **n8n**
  - Follow these short introductions to n8n workflows:
    * Data downloading (import it with the context menu on the top right of the workflow page (…) - `Import from File...` or `Import from URL...`):
      1. Create an account on n8n.cloud
      2. Create a new workflow
      3. Add a new node and select "HTTP Request"
      4. Configure the node to make a GET request to `https://minio.seriousbenentertainment.o` `%202025.csv`
      5. Add a new node and select "Write Binary File"
      6. Configure the node to write the data to your local disk
    * Mediapipe (import it with the context menu on the top right of the workflow page (…) - `Import from File...` or `Import from URL...`):
      1. Create a new workflow
      2. Add a new node and select "HTTP Request" - configure the node to make a GET request to `https://minio.seriousbenentertainment.org:9000/data/input.mp4`
      3. Add a new node Extract from File
      4. Add a new node Build JSON
      5. Add a new node and select "MediaPipe"
      6. Add a new node HTTP Request - configure the node to make a POST request to `http://212.227.102.172:8000/mediapipe`

7. Add a new node Github - configure the node to commit the results to the Github repository

* In late 2025 there was a free n8n certification course added to the n8n documentation. The students can achieve a certificate of completion for this course as part of this class (see an example Certificate of Completion).

- **Streamlit on Snowflake**
  - Follow these short introductions to Snowflake:
    1. Create a free trial account on Snowflake
    2. Choose the data tutorial and follow the steps.
- **QGIS**
- Follow these short introductions to QGIS:

1. Download and install QGIS from here
2. Download the sample data from:
   - here and unzip them into one folder - it needs to contain all these files and sub-folders: `Day2Data Day4Data Malawi-healthsites mw_districts mw_shp Malawi.qgz`
3. Open QGIS and open the project file `Malawi.qgs`
4. Open Department of Forestry website and download some or all of the zip files.
5. Add the layers via the menu `Layer` -> `Add layer` -> `Add Vector Layer` and select the shapefiles from the unzipped folders.

- **Tableau**
  - Follow these short introductions to Tableau:
    1. Download and install Tableau Desktop which is free for students or use Tableau Public online.
    2. Download the sample data from here and open it in Tableau.
- **Power BI**
  - Follow these short introductions to Power BI:
    1. We are using Power BI within MS Teams (it would also be available here to use as Power BI Desktop Application).
    2. Open an existing Power BI project and make yourself familiar with the interface.
- **KNIME**
  - Follow these short introductions to KNIME:
    1. Download and install KNIME Analytics Platform from here
    2. Create a new KNIME workflow and add a CSV Reader node to read data from a CSV file. Configure the node to use Custom/KNIME URL and add `https://raw.githubusercontent.com/datasciencedojo/datasets/refs/heads/master/ti` in the URL field. Configure the node to properly read column headers and data types.
    3. Explore the Data - Add a "Statistics" node to understand the data distribution - Use the "Missing Value" node to identify missing values in columns like Age, Cabin, and Embarked

4. Handle Missing Values
    * For the Age column: use “Missing Value” node with mean or median imputation
    * For the Cabin column: create a new binary feature indicating whether cabin information is available
    * For the Embarked column: use mode imputation or remove the few rows with missing values
5. Data Transformation
    * Use “Column Filter” to remove unnecessary columns (e.g., PassengerId, Name, Ticket)
    * Apply “String Manipulation” for categorical encoding
    * Use “Normalizer” node for numerical features if needed
6. Export Cleaned Data
    * Use “CSV Writer” node to save the cleaned dataset
    * Verify the output and check data quality

2 Descriptive statistics

Descriptive statistics summarizes and presents the main features of a dataset so you can understand what the data look like before modeling or inference. It organizes raw values into clear numerical summaries and visuals, without making probabilistic claims about a wider population. In analytics projects, this first pass helps you validate data quality, spot outliers, and communicate patterns to stakeholders.

See Types of Data and Types of Variables to build an data wrangling foundation for descriptive statistics.

What we typically summarize:

2.0.1 1. Central Tendency

Central tendency measures identify the typical or central value in a dataset.

- The **mean** (arithmetic average) is sensitive to extreme values, making it suitable for symmetric distributions.
- The **median** (middle value when sorted) is robust to outliers and preferred for skewed data.
- The **mode** (most frequent value) is useful for categorical data and can identify the most common category or value. In business analytics, choosing the appropriate measure depends on data distribution and the presence of outliers.

To set up the working directory use `getwd()` to show the current and `setwd()` to set a new working directory. For help, use the commands `?`, `??` and `help` to read the help pages:

```
# Getting the current working directory
getwd()

# Setting working directory (example for MacOS)
setwd("/Users/<username>/Documents/Analytical-Skills-for-Business")

# Opening help pages for functions can be done in multiple ways, for example:
# Using the `?` operator
?<function_name>
# or  using the `help` function
help(<function_name>)
# and sometimes you need to use the `??` operator to search for documentation
??<search_term>
```

R is an functional programming language used by many data analysts for their daily statistical work. It has a rich ecosystem of packages specifically designed for data cleaning tasks. But it is also usable as a calculation engine, see this example for a simple mathematical operation like:

$$\sqrt{81} - 2^2 \cdot \frac{1}{4}$$

```r
# Reading the help pages for `sqrt` function
?sqrt

# Calculation the example
result <- sqrt(81)-2^{2}*1/4

# Printing the result
print(result)
```

```
[1] 8
```

and it is simple to assign data to variables, like in this example:

```r
# Loading necessary libraries
library("tidyverse")

# Reading the help pages for `tibble` function
# (choose the right information in the RStudio Help tab / panel)
??tibble

# Creating a simple data frame
state <- c("NRW", "HH", "HB")
cases <- c(628, 98, 50)
df_covid <- tibble(state, cases)

# Displaying the data frame
print(df_covid)
```

```
# A tibble: 3 x 2
  state cases
  <chr> <dbl>
1 NRW     628
2 HH       98
3 HB       50
```

After these R foundation basics we can calculate the central tendency measures:

```r
# Loading necessary libraries
library(tidyverse)

# Creating a sample dataset
sample_data <- tibble(
  values = c(1, 2, 2, 3, 4, 5, 100)  # Example data with an outlier
)

# Reading the help pages for `mean` function
help(mean)

# Calculating central tendency measures
```

```r
central_tendency <- sample_data %>%
  summarise(
    mean = mean(values),
    median = median(values),
    mode = as.numeric(names(sort(table(values), decreasing = TRUE)[1]))
  )

# Printing the results
print(central_tendency)
```

```
# A tibble: 1 x 3
   mean median  mode
  <dbl>  <dbl> <dbl>
1  16.7      3     2
```

2.0.2 2. Variability

Variability measures quantify data spread around the center.

- The **range** (maximum minus minimum) provides a simple spread indicator but is sensitive to outliers.
- The **Variance** measures average squared deviations from the mean, while
- The **standard deviation** (square root of variance) expresses spread in original units, facilitating interpretation.
- The **interquartile range (IQR)** (difference between 75th and 25th percentiles) is robust to outliers and describes middle 50% spread. In business contexts, variability indicates risk, consistency, or process stability.

```r
# Calculating variability measures
variability <- sample_data %>%
  summarise(
    range = max(values) - min(values),
    variance = var(values),
    sd = sd(values),
    iqr = IQR(values)
  )

# Printing the results
print(variability)
```

```
# A tibble: 1 x 4
  range variance    sd   iqr
  <dbl>    <dbl> <dbl> <dbl>
1    99    1351.  36.8   2.5
```

2.0.3 3. Distribution Shape

Distribution shape characteristics reveal asymmetry and tail behavior.

- The **Skewness** measures distribution asymmetry: positive skewness indicates a right tail (mean > median), negative skewness indicates a left tail (mean < median), and zero indicates symmetry.
- The **Kurtosis** measures tail heaviness and peakedness: high kurtosis indicates heavy tails with more outliers, low kurtosis indicates light tails.

These measures help identify data transformations needed and potential outliers in business analytics.

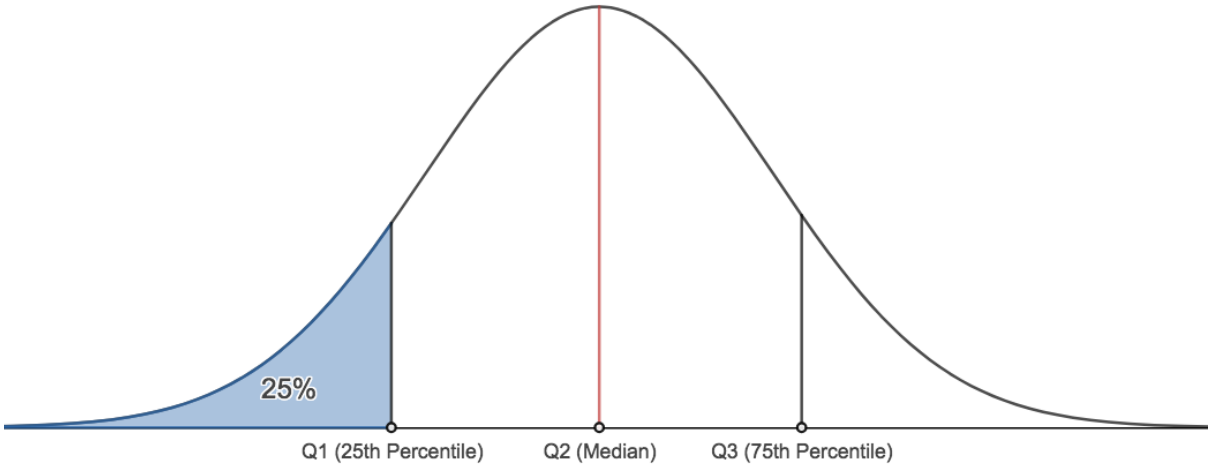
```
# Loading moments package for skewness and kurtosis
library(tidyverse)
library(moments)

# Calculating distribution shape measures
distribution_shape <- sample_data %>%
  summarise(
    skewness = skewness(values),
    kurtosis = kurtosis(values)
  )

# Printing the results
print(distribution_shape)
```

```
# A tibble: 1 x 2
  skewness kurtosis
     <dbl>    <dbl>
1     2.04     5.15
```

2.0.4 4. Frequencies and Percentiles

Frequency analysis counts occurrences and calculates proportions, revealing data distribution patterns.

- **Percentiles** (quantiles) divide data into equal parts: quartiles (25%, 50%, 75%), deciles (10% intervals), or any custom percentile.



These measures identify data position and help detect outliers. In business, percentiles benchmark performance (e.g., top 10% customers) and support decision thresholds.

```
# Loading moments package for skewness and kurtosis
library(tidyverse)

# Calculating frequencies and percentiles
frequencies <- sample_data %>%
  count(values) %>%
  mutate(proportion = n / sum(n))

# Printing frequency table
print(frequencies)
```

```
# A tibble: 6 x 3
  values     n proportion
   <dbl> <int>      <dbl>
1      1     1      0.143
2      2     2      0.286
3      3     1      0.143
4      4     1      0.143
5      5     1      0.143
6    100     1      0.143
```

```
# Calculating various percentiles
percentiles <- sample_data %>%
  summarise(
    q25 = quantile(values, 0.25),
    q50 = quantile(values, 0.50),  # median
    q75 = quantile(values, 0.75),
    q90 = quantile(values, 0.90),
    q95 = quantile(values, 0.95)
  )

# Printing percentiles
print(percentiles)
```

```
# A tibble: 1 x 5
    q25   q50   q75   q90   q95
  <dbl> <dbl> <dbl> <dbl> <dbl>
1     2     3   4.5  43.0  71.5
```

What are common methods to summarize:

2.0.5 5. Numerical Methods

Numerical methods systematically compute summary statistics and organize data into interpretable structures.

- The **Summary metrics** condense datasets into key statistics (mean, median, standard deviation, quartiles), enabling quick assessment.
- The **Frequency tables** tabulate value counts or binned ranges, revealing distribution patterns. These methods form the foundation for exploratory data analysis and inform subsequent statistical modeling in business analytics.

```r
# Loading necessary libraries
library(tidyverse)

# Creating a more comprehensive summary
summary_metrics <- sample_data %>%
  summarise(
    count = n(),
    min = min(values),
    q1 = quantile(values, 0.25),
    median = median(values),
    mean = mean(values),
    q3 = quantile(values, 0.75),
    max = max(values),
    sd = sd(values),
    variance = var(values)
  )

# Printing summary metrics
print(summary_metrics)
```

```
# A tibble: 1 x 9
  count   min    q1 median  mean    q3   max    sd variance
  <int> <dbl> <dbl>  <dbl> <dbl> <dbl> <dbl> <dbl>    <dbl>
1     7     1     2      3  16.7   4.5   100  36.8    1351.
```

```r
# Creating a frequency table with binned ranges
binned_data <- sample_data %>%
  mutate(bin = cut(values, breaks = c(0, 2, 4, 6, Inf),
                   labels = c("0-2", "2-4", "4-6", ">6"))) %>%
  count(bin) %>%
  mutate(percentage = n / sum(n) * 100)
```

```r
# Printing frequency table
print(binned_data)
```

```
# A tibble: 4 x 3
  bin       n percentage
  <fct> <int>      <dbl>
1 0-2       3       42.9
2 2-4       2       28.6
3 4-6       1       14.3
4 >6        1       14.3
```

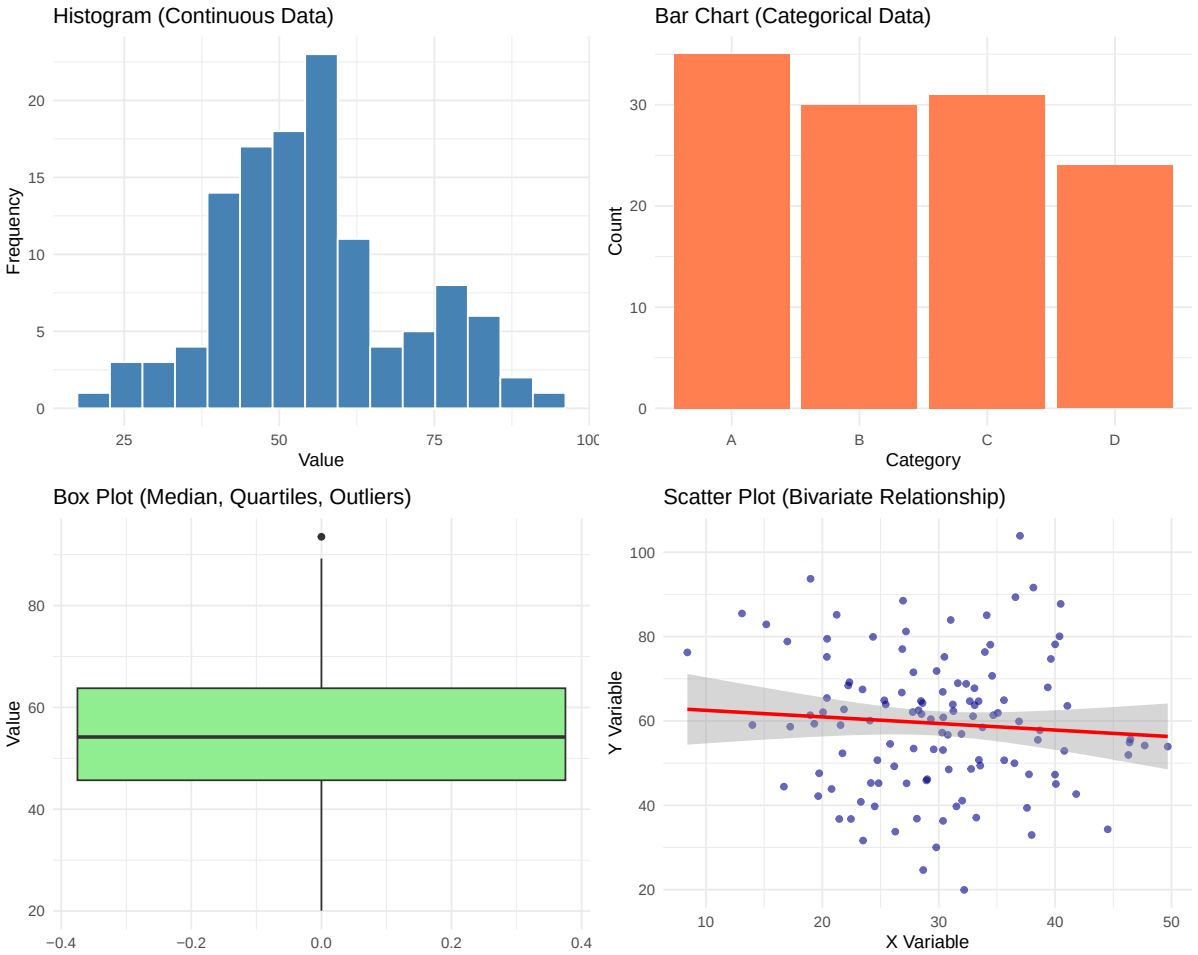
2.0.6 6. Graphical Methods

Graphical methods visualize data distributions and relationships, making patterns immediately recognizable.

- **Histograms** display continuous data frequency distributions across bins.
- **Bar and pie charts** compare categorical data frequencies or proportions.
- **Box plots** show median, quartiles, and outliers, facilitating distributional comparisons.
- **Scatter plots** reveal bivariate relationships and correlation patterns. In business analytics, visualizations communicate insights to non-technical stakeholders and guide data-driven decisions.

```r
# Loading necessary libraries
library(tidyverse)
library(gridExtra)
library(ggplot2)

# Creating a larger dataset for better visualizations
set.seed(42)
large_data <- tibble(
  continuous_var = c(rnorm(100, mean = 50, sd = 10), rnorm(20, mean = 80, sd = 5)),
  category = sample(c("A", "B", "C", "D"), 120, replace = TRUE),
  x_var = rnorm(120, mean = 30, sd = 8),
  y_var = 2 * rnorm(120, mean = 30, sd = 8) + rnorm(120, mean = 0, sd = 5)
)

# Creating multiple plots
library(gridExtra)

# Histogram for continuous data
p1 <- ggplot(large_data, aes(x = continuous_var)) +
  geom_histogram(bins = 15, fill = "steelblue", color = "white") +
  labs(title = "Histogram (Continuous Data)", x = "Value", y = "Frequency") +
  theme_minimal()

# Bar chart for categorical data
p2 <- ggplot(large_data, aes(x = category)) +
  geom_bar(fill = "coral") +
  labs(title = "Bar Chart (Categorical Data)", x = "Category", y = "Count") +
```

```r
  theme_minimal()

# Box plot
p3 <- ggplot(large_data, aes(y = continuous_var)) +
  geom_boxplot(fill = "lightgreen") +
  labs(title = "Box Plot (Median, Quartiles, Outliers)", y = "Value") +
  theme_minimal()

# Scatter plot for bivariate relationship
p4 <- ggplot(large_data, aes(x = x_var, y = y_var)) +
  geom_point(color = "darkblue", alpha = 0.6) +
  geom_smooth(method = "lm", color = "red", se = TRUE) +
  labs(title = "Scatter Plot (Bivariate Relationship)", x = "X Variable", y = "Y Variable") +
  theme_minimal()

# Arranging plots in a grid
grid.arrange(p1, p2, p3, p4, ncol = 2)
```



Business analytics context:

- For monthly revenue by region, the mean signals typical performance, the standard deviation shows volatility, a box plot quickly flags outliers, and a bar chart compares regions. These summaries guide prioritization (e.g., regions with high variability may require deeper investigation) and set baselines for forecasting and experimentation.

- For an accessible overview of types, methods, and examples, see ResearchMethod.net.

2.1 Data Types: Univariate, Bivariate, and Multivariate

Data types in business analytics can be classified based on the number of variables involved in the analysis. Understanding these classifications helps in selecting appropriate analytical techniques and visualizations.

2.1.1 Univariate data

Univariate analysis examines one variable at a time to understand its distribution, central tendency, and variability. It answers questions like "What is typical?" and "How much variation exists?" Common techniques include summary statistics (mean, median, mode, standard deviation), frequency distributions, and visualizations such as histograms, box plots, and bar charts. **Business example:** Analyzing monthly sales revenue to identify typical performance and outliers.

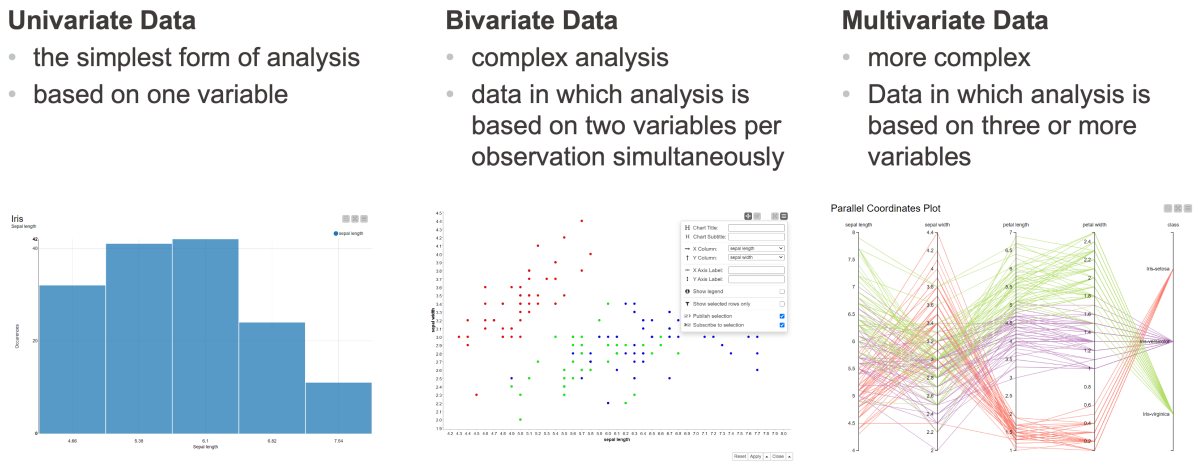
2.1.2 Bivariate data

Bivariate analysis examines relationships between two variables to identify associations, correlations, or dependencies. It answers questions like "How do these variables relate?" and "Does one variable predict another?" Common techniques include correlation coefficients (Pearson, Spearman), cross-tabulation, simple linear regression, and visualizations such as scatter plots and grouped bar charts. **Business example:** Examining the relationship between advertising spend and sales revenue to determine campaign effectiveness.

2.1.3 Multivariate data

Multivariate analysis examines three or more variables simultaneously to understand complex relationships, interactions, and patterns. It answers questions like "How do multiple factors jointly influence outcomes?" and "What hidden patterns exist?" Common techniques include multiple regression, principal component analysis (PCA), cluster analysis, and visualizations such as correlation matrices and heatmaps. **Business example:** Analyzing customer demographics, purchase history, and behavioral data simultaneously to identify market segments and predict customer lifetime value.

Figure 5: **Source:** *KNIME*



2.2 Techniques for Scores, Rankings, Metrics, and Composite Indicators

Performance measurement in business analytics requires systematic approaches to quantify complex phenomena through structured numerical representations.

2.2.1 Constructing

Construction involves designing and calculating meaningful measures from raw data. This includes:

- **Score development**: Creating numerical values that represent performance levels (e.g., credit scores, customer satisfaction scores)
- **Ranking systems**: Ordering entities based on specific criteria (e.g., sales performance rankings, market share positions)
- **Metric formulation**: Defining key performance indicators (KPIs) that align with business objectives (e.g., conversion rates, ROI, customer lifetime value)
- **Composite indicators**: Combining multiple metrics into single summary measures (e.g., balanced scorecards, economic indices, sustainability ratings)

Business example: Constructing a customer health score by combining purchase frequency, average order value, and engagement metrics with weighted coefficients.

2.2.2 Interpreting

Interpretation translates numerical results into actionable business insights. This requires:

- **Contextualization**: Understanding what scores mean relative to benchmarks, targets, or historical performance
- **Trend analysis**: Identifying patterns and changes over time
- **Comparative analysis**: Assessing performance relative to competitors or industry standards
- **Threshold identification**: Determining critical values that trigger decisions or actions

Business example: Interpreting a Net Promoter Score (NPS) of 45 as "good" for the retail industry, but identifying a downward trend requiring investigation.

2.2.3 Evaluating

Evaluation assesses the quality, validity, and effectiveness of measurement systems. Key considerations include:

- **Validity**: Does the measure capture what it intends to measure?
- **Reliability**: Are results consistent and reproducible?
- **Sensitivity**: Does the metric detect meaningful changes?
- **Actionability**: Can insights drive concrete business decisions?
- **Bias assessment**: Identifying and mitigating systematic errors or unfair representations

Business example: Evaluating whether employee performance scores fairly represent actual contributions or reflect demographic biases, leading to refined evaluation criteria.

2.3 Visualizing and Exploration of Data Types

Effective data visualization transforms raw data into visual representations that reveal patterns, trends, and insights. Different data types require specific visualization approaches and exploratory techniques.

2.3.1 Categorical Data

Categorical data represents discrete groups or categories without inherent numerical ordering (nominal) or with meaningful ordering (ordinal). Visualization and exploration focus on frequency distributions and group comparisons.

```
# Loading necessary libraries
library(tidyverse)

# Creating a sample nominal categorical dataset
nominal_data <- tibble(
  category = sample(c("B", "C", "A", "D"), 10, replace = TRUE)
)

# Creating a sample ordinal categorical dataset
ordinal_data <- tibble(
  rating = factor(sample(c("Excellent", "Good", "Poor", "Fair", "Very Good"), 10,
                  replace = TRUE),
                  levels = c("Poor", "Fair", "Good", "Very Good", "Excellent"),
                  ordered = TRUE)
)

# Displaying the nominal and ordinal data
print(nominal_data)
```

```
# A tibble: 10 x 1
   category
   <chr>
 1 D
 2 A
```

```
 3 A
 4 D
 5 A
 6 A
 7 B
 8 A
 9 C
10 C
```

```
print(ordinal_data)
```

```
# A tibble: 10 x 1
   rating
   <ord>
 1 Very Good
 2 Excellent
 3 Very Good
 4 Good
 5 Excellent
 6 Fair
 7 Good
 8 Very Good
 9 Poor
10 Good
```

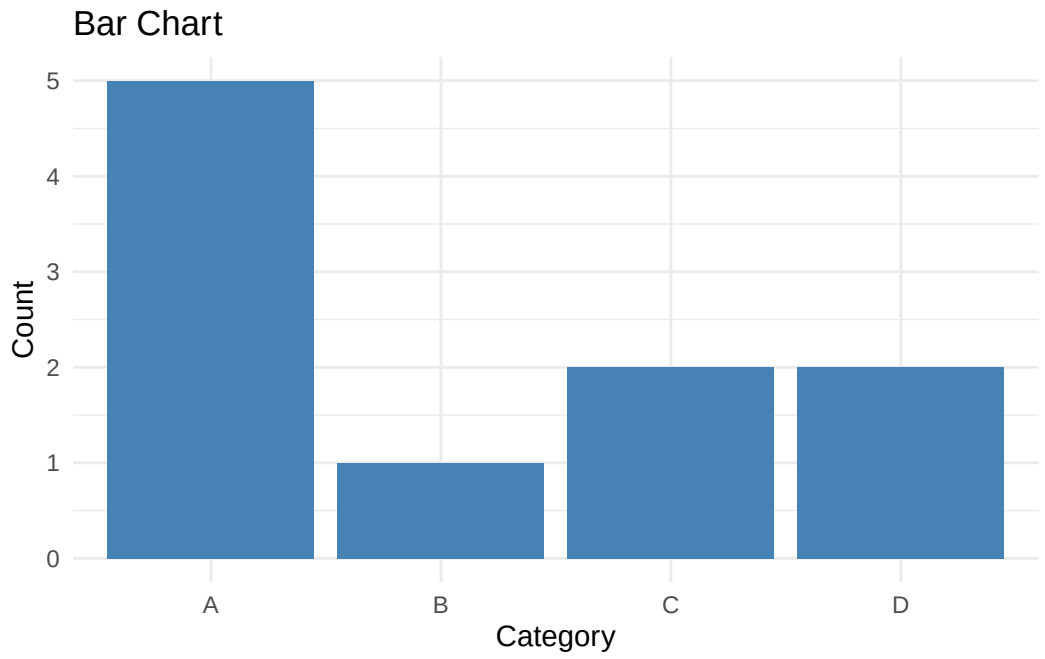
```
# Displaying ordered levels of ordinal data
levels(ordinal_data$rating)
```

```
[1] "Poor"      "Fair"      "Good"      "Very Good" "Excellent"
```

Common visualizations:

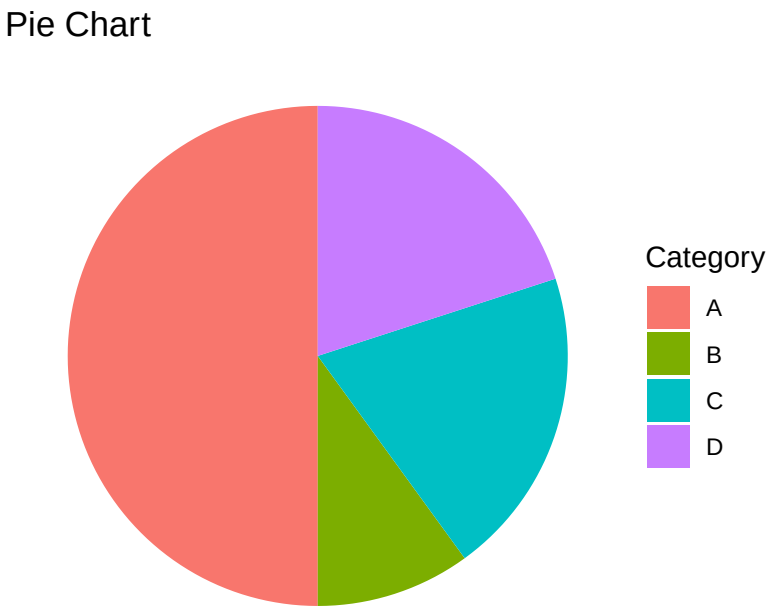
- **Bar charts**: Compare frequencies or proportions across categories

```
library(ggplot2)
ggplot(nominal_data, aes(x = category)) +
  geom_bar(fill = "steelblue") +
  labs(title = "Bar Chart", x = "Category", y = "Count") +
  theme_minimal()
```



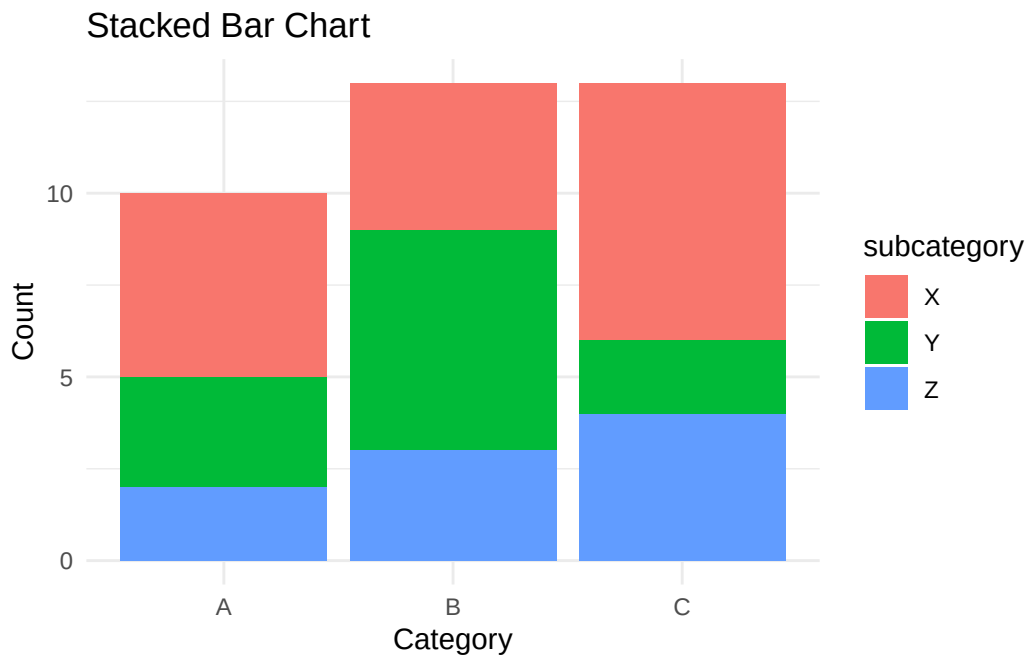
- **Pie charts**: Show part-to-whole relationships for categorical breakdowns

```
library(ggplot2)
category_counts <- as.data.frame(table(nominal_data$category))
ggplot(category_counts, aes(x = "", y = Freq, fill = Var1)) +
  geom_bar(stat = "identity", width = 1) +
  coord_polar("y") +
  labs(title = "Pie Chart", fill = "Category") +
  theme_void()
```



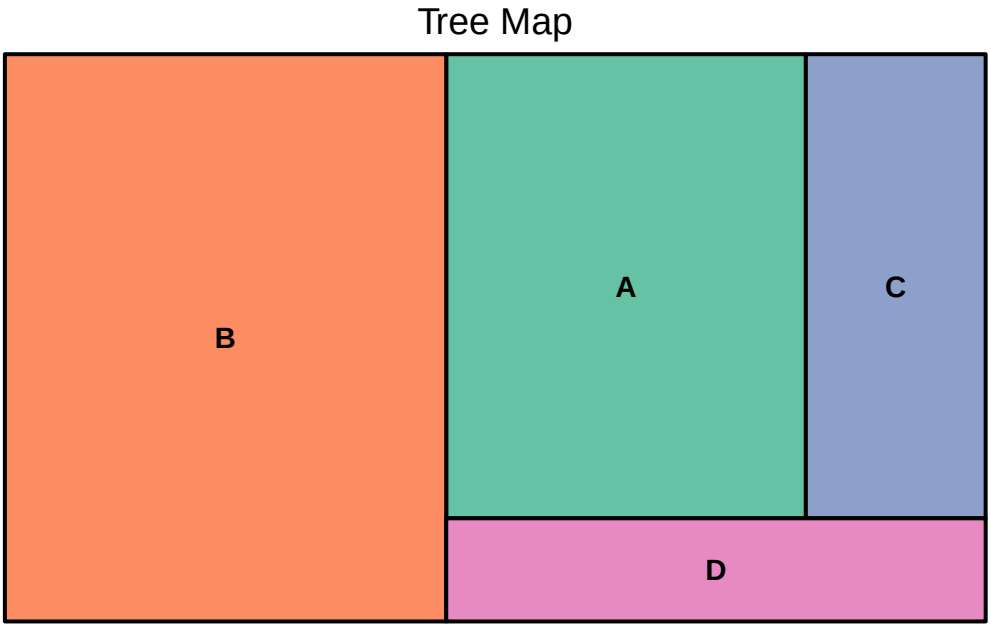
- **Stacked bar charts**: Display multiple categorical variables simultaneously

```r
library(ggplot2)
combined_cat_data <- data.frame(
  category = rep(c("A", "B", "C"), each = 3),
  subcategory = rep(c("X", "Y", "Z"), 3),
  count = c(5, 3, 2, 4, 6, 3, 7, 2, 4)
)
ggplot(combined_cat_data, aes(x = category, y = count, fill = subcategory)) +
  geom_bar(stat = "identity") +
  labs(title = "Stacked Bar Chart", x = "Category", y = "Count") +
  theme_minimal()
```



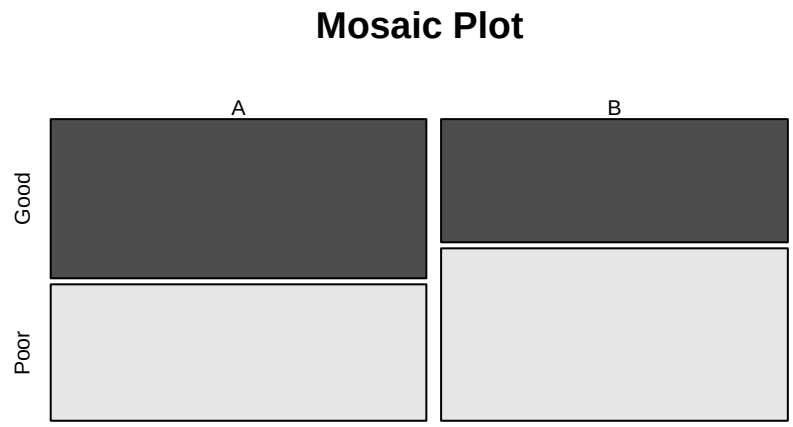
- **Tree maps**: Visualize hierarchical categorical data with nested rectangles

```r
library(treemap)
treemap_data <- data.frame(
  category = c("A", "B", "C", "D"),
  value = c(30, 45, 15, 10)
)
treemap(treemap_data, index = "category", vSize = "value",
        title = "Tree Map", palette = "Set2")
```



- **Mosaic plots**: Show relationships between multiple categorical variables

```
mosaic_data <- data.frame(
  category = sample(c("A", "B"), 50, replace = TRUE),
  rating = sample(c("Good", "Poor"), 50, replace = TRUE)
)
mosaicplot(table(mosaic_data$category, mosaic_data$rating),
           main = "Mosaic Plot", color = TRUE)
```



Exploratory techniques:

- Frequency tables and cross-tabulations
- Mode identification

- Chi-square tests for independence
- Contingency analysis

Business example: Visualizing customer segments by region using bar charts, analyzing product category preferences with pie charts, or exploring the relationship between customer type and purchase channel using mosaic plots.

2.3.2 Numerical Data

Numerical data consists of quantitative measurements on continuous or discrete scales. Visualization emphasizes distributions, central tendencies, variability, and relationships between variables.

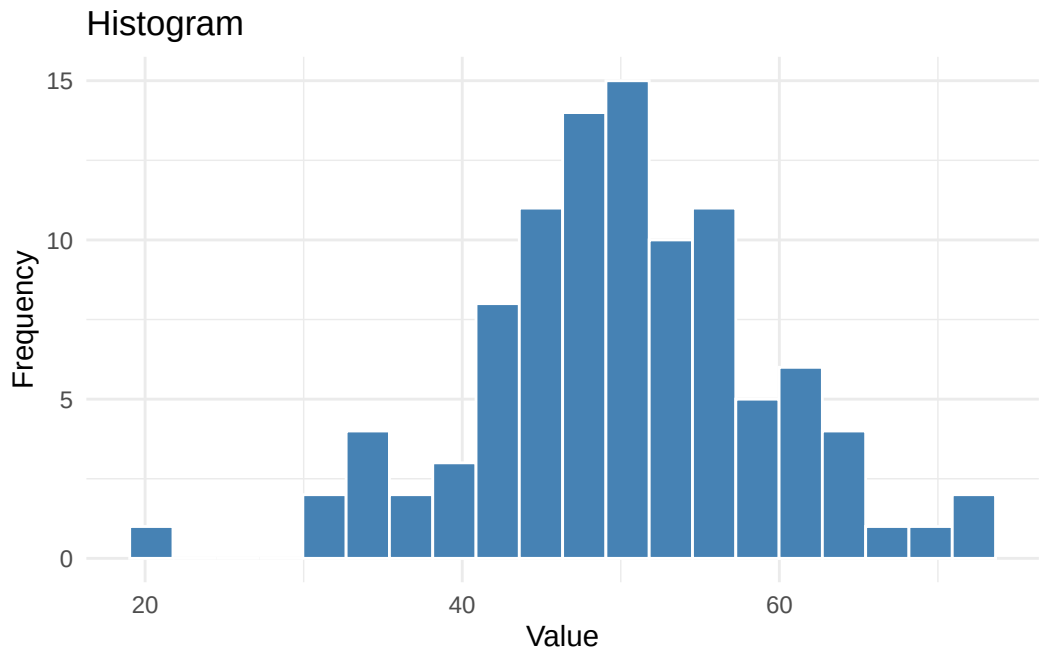
```
# Creating a sample numerical dataset
numerical_data <- tibble(
  value1 = rnorm(100, mean = 50, sd = 10),
  value2 = rnorm(100, mean = 30, sd = 5)
)

# Displaying the numerical data
print(numerical_data)
```

```
# A tibble: 100 x 2
   value1 value2
    <dbl>  <dbl>
 1   48.3   21.8
 2   40.9   27.7
 3   63.9   26.0
 4   50.1   32.3
 5   54.3   28.8
 6   70.7   28.1
 7   47.0   27.3
 8   51.2   29.5
 9   47.5   22.0
10   60.3   35.0
# i 90 more rows
```

Common visualizations:

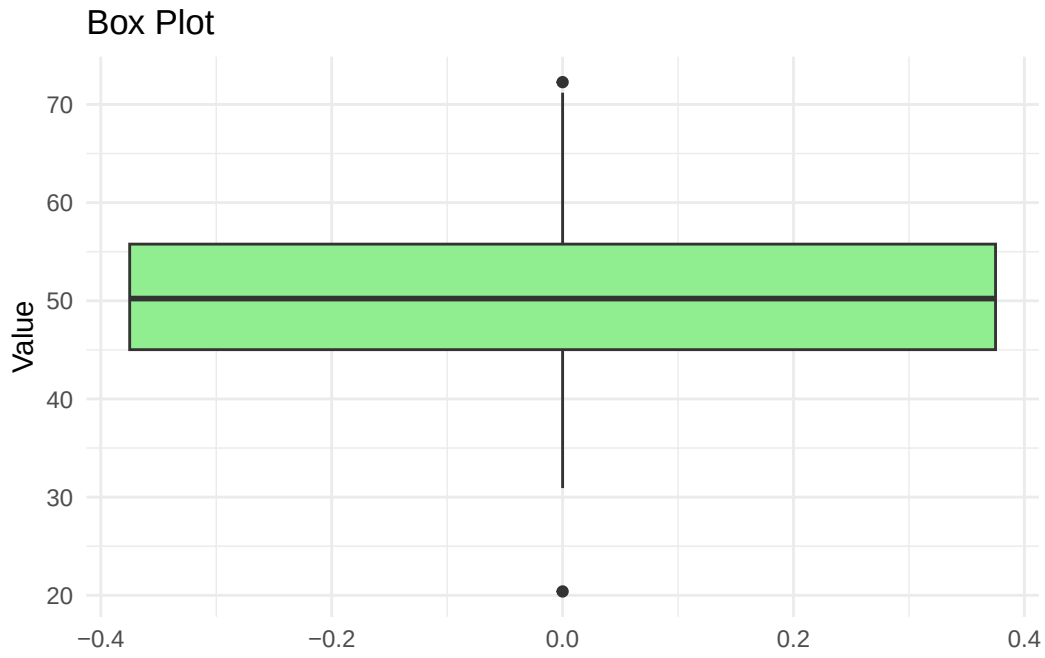
- **Histograms**: Display distribution of continuous variables

```
library(ggplot2)
ggplot(numerical_data, aes(x = value1)) +
  geom_histogram(bins = 20, fill = "steelblue", color = "white") +
  labs(title = "Histogram", x = "Value", y = "Frequency") +
  theme_minimal()
```



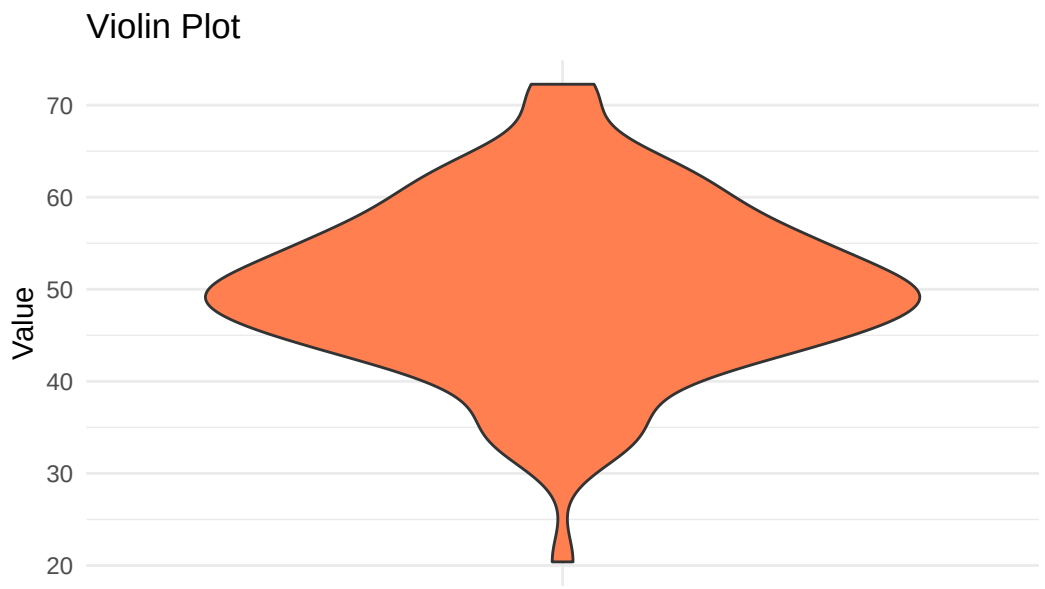
- **Box plots**: Show median, quartiles, and outliers for comparison

```
library(ggplot2)
ggplot(numerical_data, aes(y = value1)) +
  geom_boxplot(fill = "lightgreen") +
  labs(title = "Box Plot", y = "Value") +
  theme_minimal()
```



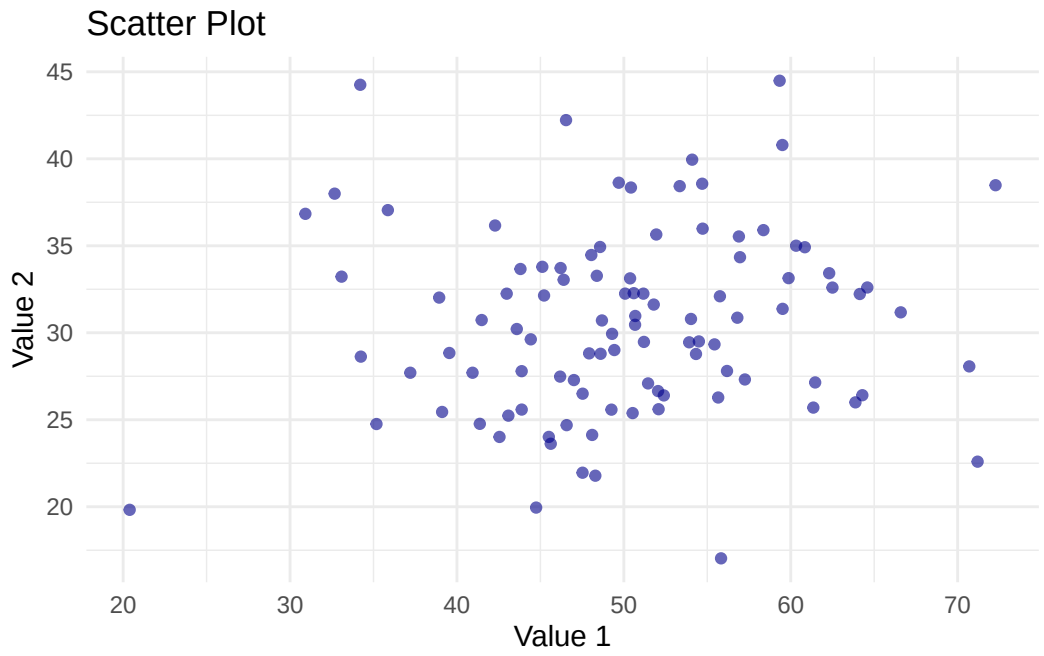
- **Violin plots**: Combine box plots with density curves

```
library(ggplot2)
ggplot(numerical_data, aes(x = "", y = value1)) +
  geom_violin(fill = "coral") +
  labs(title = "Violin Plot", x = "", y = "Value") +
  theme_minimal()
```



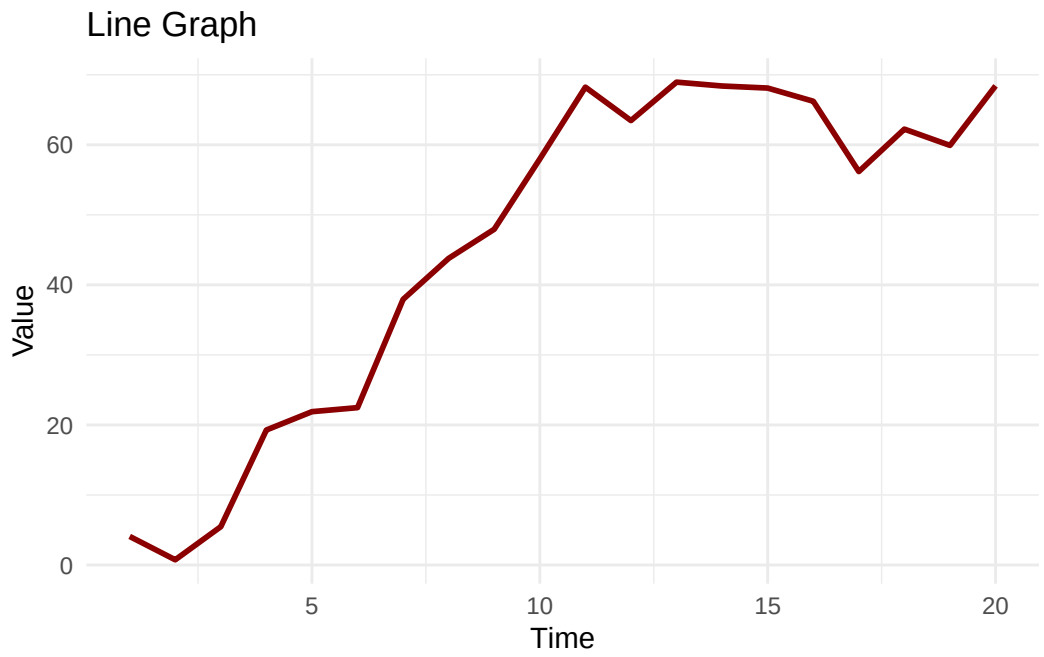
- **Scatter plots**: Reveal relationships between two numerical variables

```
library(ggplot2)
ggplot(numerical_data, aes(x = value1, y = value2)) +
  geom_point(color = "darkblue", alpha = 0.6) +
  labs(title = "Scatter Plot", x = "Value 1", y = "Value 2") +
  theme_minimal()
```



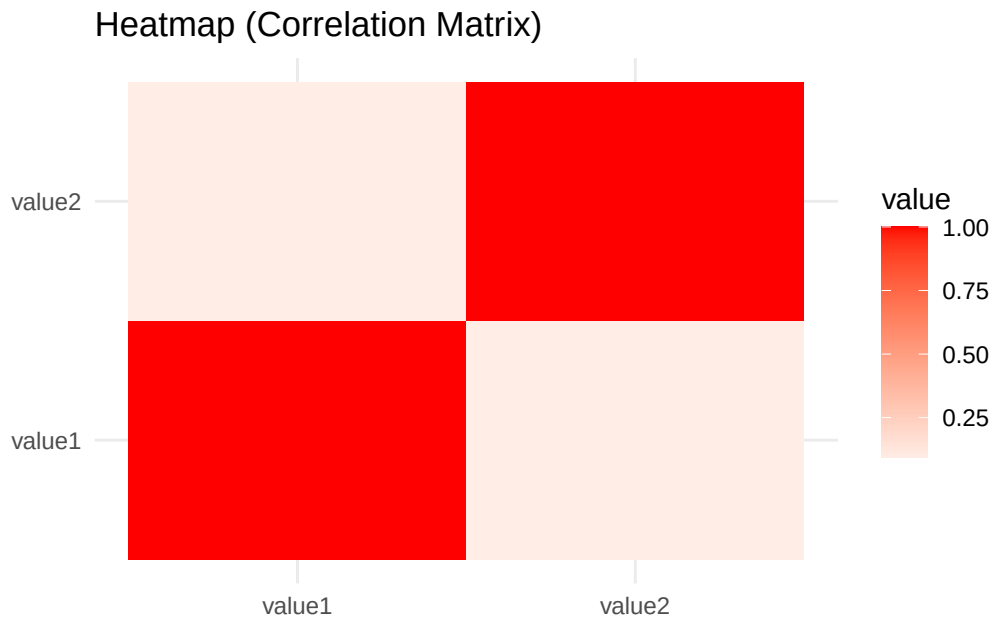
- **Line graphs**: Track changes over ordered sequences

```
library(ggplot2)
time_data <- data.frame(
  time = 1:20,
  value = cumsum(rnorm(20, mean = 2, sd = 5))
)
ggplot(time_data, aes(x = time, y = value)) +
  geom_line(color = "darkred", linewidth = 1) +
  labs(title = "Line Graph", x = "Time", y = "Value") +
  theme_minimal()
```



- **Heatmaps**: Display correlation matrices or intensity patterns

```
library(ggplot2)
library(reshape2)
cor_matrix <- cor(numerical_data)
cor_melted <- melt(cor_matrix)
ggplot(cor_melted, aes(x = Var1, y = Var2, fill = value)) +
  geom_tile() +
  scale_fill_gradient2(low = "blue", mid = "white", high = "red", midpoint = 0) +
  labs(title = "Heatmap (Correlation Matrix)", x = "", y = "") +
  theme_minimal()
```



Exploratory techniques:

- Summary statistics (mean, median, standard deviation)
- Distribution analysis (skewness, kurtosis)
- Outlier detection
- Correlation analysis
- Trend identification

Business example: Using histograms to analyze revenue distributions, box plots to compare sales performance across quarters, scatter plots to examine the relationship between marketing spend and customer acquisition, or heatmaps to visualize product sales correlations.

2.3.3 Time Series Data

Time series data represents sequential observations measured at successive time points. Visualization and exploration focus on temporal patterns, trends, seasonality, and cyclical behaviors.

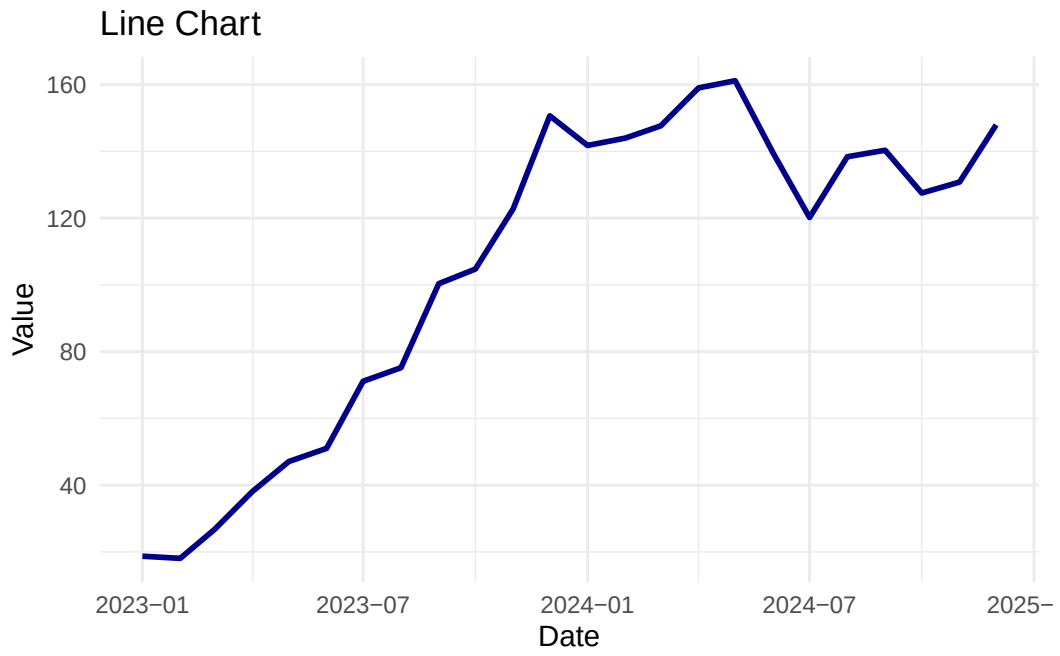
```r
# Creating a sample time series dataset
set.seed(42)
time_series_data <- tibble(
  date = seq.Date(from = as.Date("2023-01-01"), by = "month", length.out = 24),
  value = cumsum(rnorm(24, mean = 5, sd = 10))
)

# Displaying the time series data
print(time_series_data)
```

```
# A tibble: 24 x 2
   date       value
   <date>     <dbl>
 1 2023-01-01  18.7
 2 2023-02-01  18.1
 3 2023-03-01  26.7
 4 2023-04-01  38.0
 5 2023-05-01  47.1
 6 2023-06-01  51.0
 7 2023-07-01  71.1
 8 2023-08-01  75.2
 9 2023-09-01 100.
10 2023-10-01 105.
# i 14 more rows
```

Common visualizations:

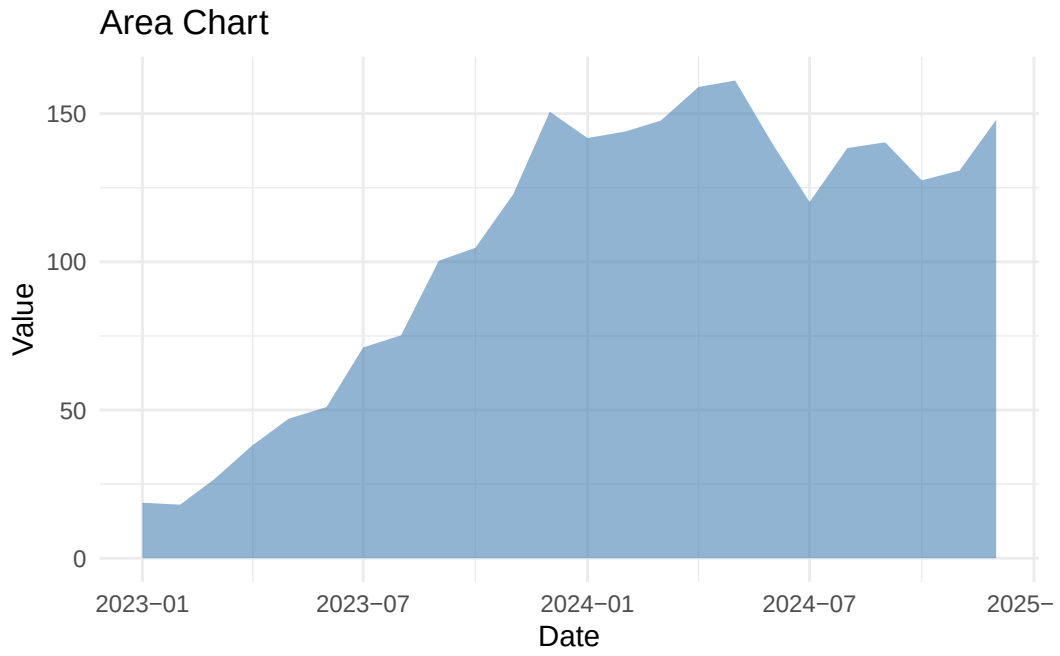
- **Line charts**: Display trends and patterns over time

```r
library(ggplot2)
ggplot(time_series_data, aes(x = date, y = value)) +
  geom_line(color = "darkblue", linewidth = 1) +
  labs(title = "Line Chart", x = "Date", y = "Value") +
  theme_minimal()
```



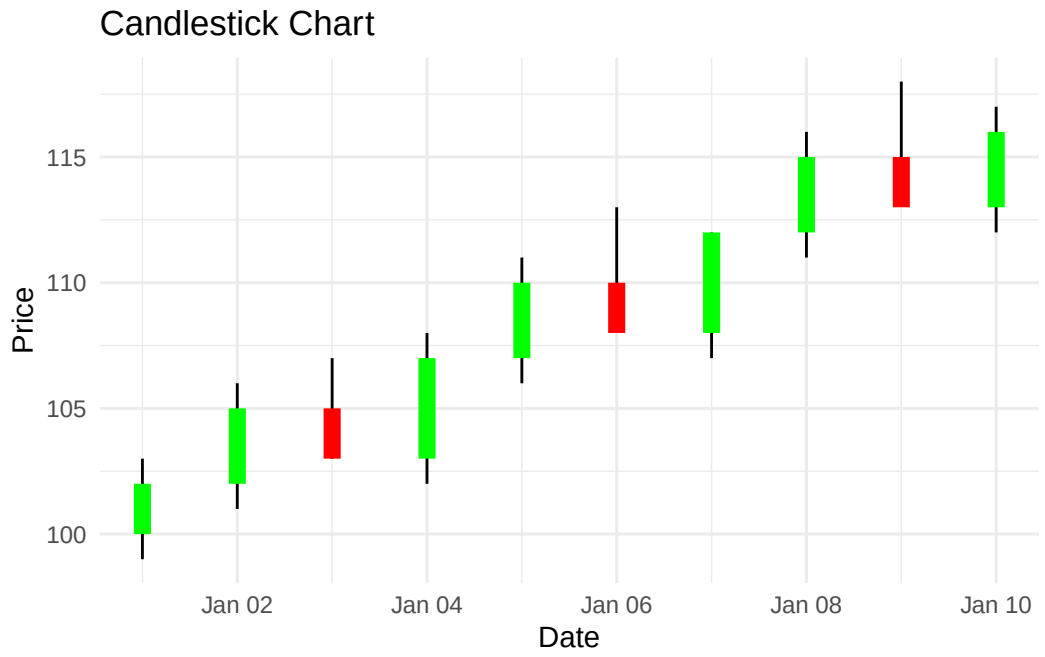
- **Area charts**: Show cumulative values or multiple series stacked

```
library(ggplot2)
ggplot(time_series_data, aes(x = date, y = value)) +
  geom_area(fill = "steelblue", alpha = 0.6) +
  labs(title = "Area Chart", x = "Date", y = "Value") +
  theme_minimal()
```



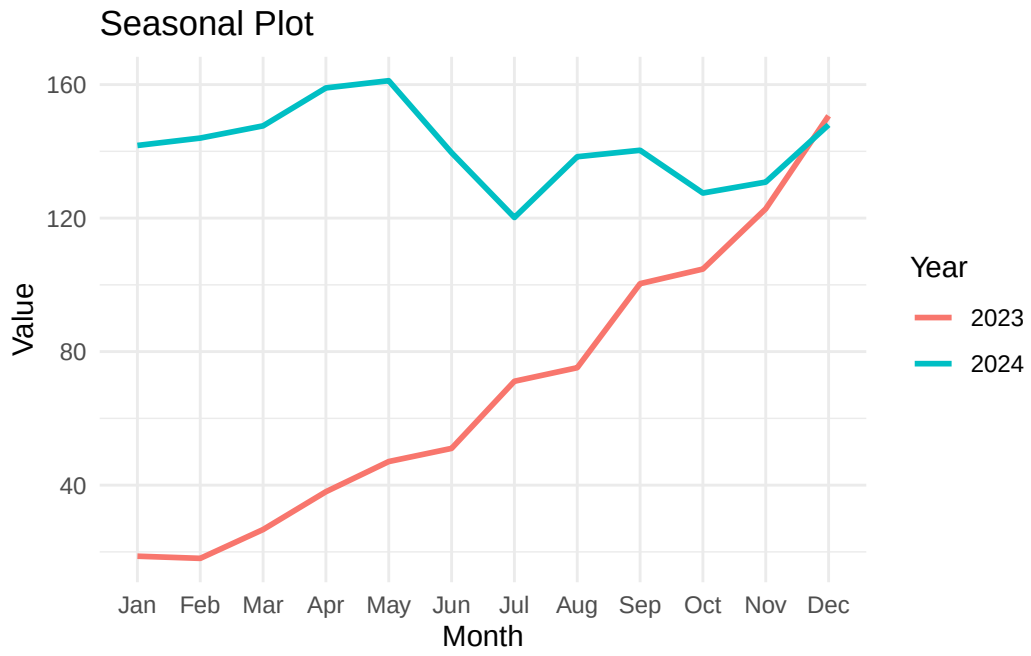
- **Candlestick charts**: Represent open, high, low, close values (financial data)

```r
library(ggplot2)
financial_data <- data.frame(
  date = seq.Date(from = as.Date("2024-01-01"), by = "day", length.out = 10),
  open = c(100, 102, 105, 103, 107, 110, 108, 112, 115, 113),
  high = c(103, 106, 107, 108, 111, 113, 112, 116, 118, 117),
  low = c(99, 101, 103, 102, 106, 109, 107, 111, 114, 112),
  close = c(102, 105, 103, 107, 110, 108, 112, 115, 113, 116)
)
ggplot(financial_data, aes(x = date)) +
  geom_segment(aes(xend = date, y = low, yend = high)) +
  geom_segment(aes(xend = date, y = open, yend = close), linewidth = 3,
               color = ifelse(financial_data$close > financial_data$open, "green", "red")) +
  labs(title = "Candlestick Chart", x = "Date", y = "Price") +
  theme_minimal()
```



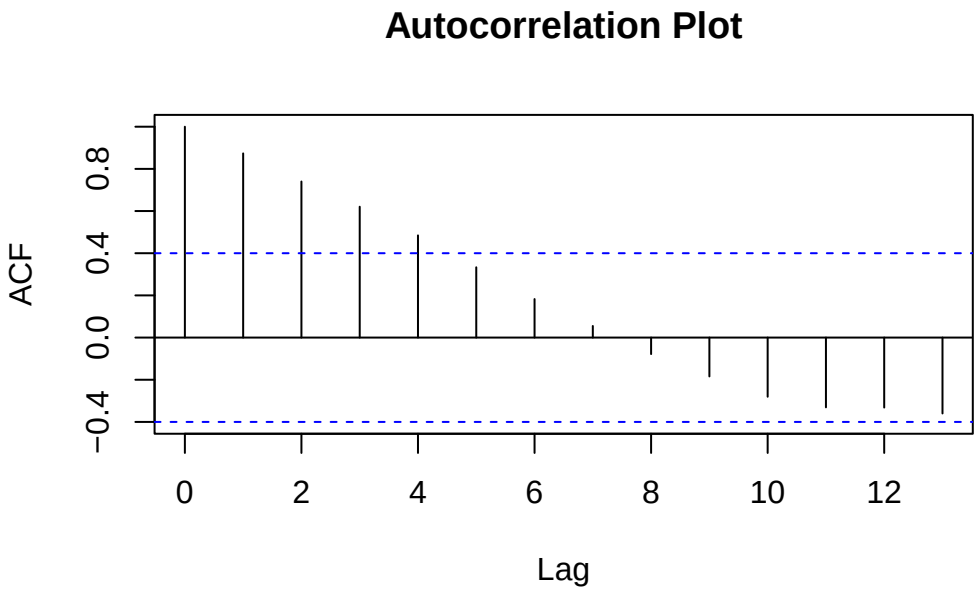
- **Seasonal plots**: Reveal recurring patterns by season or period

```r
library(ggplot2)
library(lubridate)
seasonal_data <- time_series_data %>%
  mutate(month = month(date, label = TRUE),
         year = year(date))
ggplot(seasonal_data, aes(x = month, y = value, group = year, color = as.factor(year))) +
  geom_line(linewidth = 1) +
  labs(title = "Seasonal Plot", x = "Month", y = "Value", color = "Year") +
  theme_minimal()
```



- **Autocorrelation plots**: Identify temporal dependencies

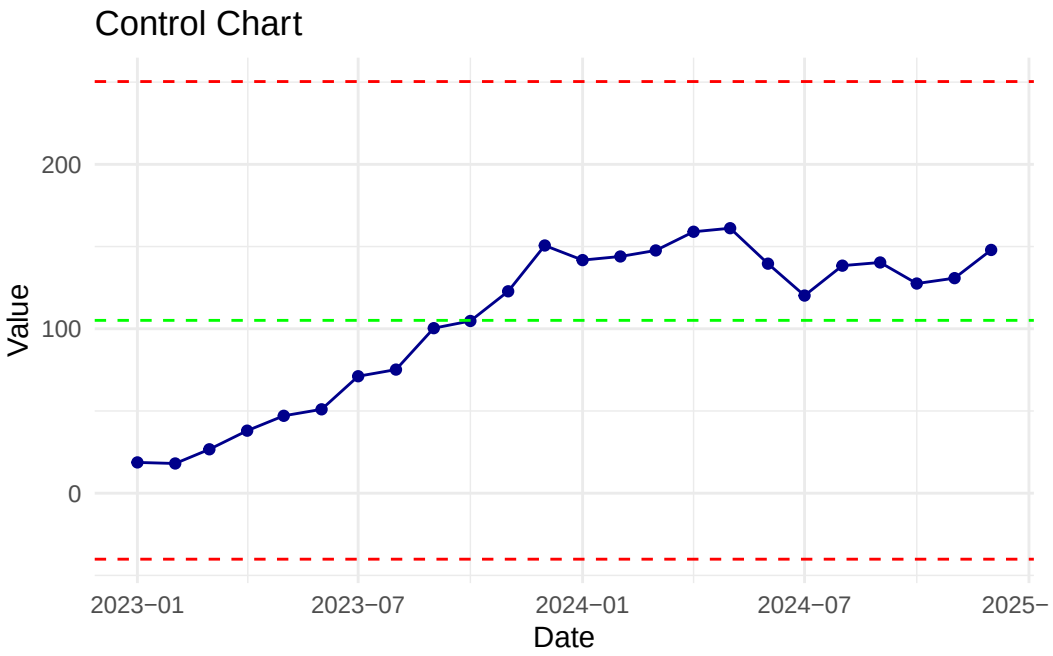
```
acf(time_series_data$value, main = "Autocorrelation Plot")
```



- **Control charts**: Monitor process stability and detect anomalies

```
library(ggplot2)
control_data <- time_series_data %>%
  mutate(mean_value = mean(value),
         ucl = mean(value) + 3 * sd(value),
         lcl = mean(value) - 3 * sd(value))
ggplot(control_data, aes(x = date, y = value)) +
```

```
  geom_line(color = "darkblue") +
  geom_point(color = "darkblue") +
  geom_hline(aes(yintercept = mean_value), color = "green", linetype = "dashed") +
  geom_hline(aes(yintercept = ucl), color = "red", linetype = "dashed") +
  geom_hline(aes(yintercept = lcl), color = "red", linetype = "dashed") +
  labs(title = "Control Chart", x = "Date", y = "Value") +
  theme_minimal()
```



Exploratory techniques:

- Trend analysis (moving averages, smoothing)
- Seasonality detection (seasonal decomposition)
- Stationarity testing
- Lag analysis and autocorrelation
- Change point detection
- Anomaly identification

Business example: Tracking monthly revenue trends with line charts, identifying seasonal sales patterns using seasonal decomposition, monitoring website traffic fluctuations with control charts, or analyzing stock price movements with candlestick charts to inform investment decisions.

2.4 Handling Messy Data

Real-world data is rarely clean or analysis-ready. Data quality issues compromise analytical accuracy and business decisions. Handling messy data involves identifying, understanding, and resolving various data quality problems through systematic cleaning and transformation processes.

2.4.1 Common Data Quality Issues

Missing Values:

- **Complete absence**: Entire records or fields are missing
- **Implicit missingness**: Represented as NULL, NA, empty strings, or placeholder values (e.g., 999, -1)
- **Patterns**: Missing Completely At Random (MCAR), Missing At Random (MAR), or Missing Not At Random (MNAR)

Handling strategies:

- Deletion (listwise or pairwise) when data is MCAR and missing percentage is low
- Imputation: mean/median/mode for numerical data, forward/backward fill for time series
- Model-based imputation: regression, k-nearest neighbors (KNN), multiple imputation
- Flag creation: adding indicator variables to preserve information about missingness

Inconsistent Data:

- **Format variations**: "New York" vs. "NY" vs. "new york"
- **Unit inconsistencies**: mixing metric and imperial measurements
- **Date format variations**: "MM/DD/YYYY" vs. "DD-MM-YYYY"
- **Encoding issues**: character encoding problems (UTF-8, ASCII)

Handling strategies:

- Standardization: converting to consistent formats and units
- Text normalization: lowercasing, trimming whitespace, removing special characters
- Date parsing with explicit format specifications
- Encoding conversion and validation

Duplicate Records:

- **Exact duplicates**: Identical records across all fields
- **Near duplicates**: Records representing the same entity with slight variations
- **Logical duplicates**: Same entity with different identifiers

Handling strategies:

- Exact match removal using unique identifiers
- Fuzzy matching for near duplicates (Levenshtein distance, Jaccard similarity)
- Record linkage and deduplication algorithms
- Master data management (MDM) approaches

Outliers and Anomalies:

- **Statistical outliers**: Values beyond expected range (e.g., 3 standard deviations from mean)
- **Domain outliers**: Values impossible or implausible in business context (negative ages, future dates)
- **Data entry errors**: Typos or transposition errors

Handling strategies:

- Detection: Z-score, IQR method, isolation forests, DBSCAN clustering
- Investigation: determine if outliers are errors or valid extreme values

- Treatment: removal, capping/winsorizing, transformation, or separate analysis
- Domain validation: implementing business rules and constraints

Structural Issues:

- **Wrong data types**: Numbers stored as text, dates as strings
- **Improper granularity**: Data too aggregated or too detailed for analysis
- **Denormalized structures**: Redundant or poorly organized data
- **Schema mismatches**: Incompatible structures when merging datasets

Handling strategies:

- Type conversion with validation
- Aggregation or disaggregation to appropriate levels
- Normalization and restructuring (wide-to-long, long-to-wide transformations)
- Schema mapping and harmonization

2.4.2 Data Cleaning Workflow

1. Profiling and Assessment:

- Examine data distributions, completeness, and patterns
- Generate data quality reports
- Identify specific issues and their prevalence

2. Validation:

- Define business rules and constraints
- Implement validation checks (range checks, format checks, logical consistency)
- Flag violations for review

3. Cleaning:

- Apply correction strategies systematically
- Document all transformations
- Maintain audit trails

4. Verification:

- Validate cleaned data against quality criteria
- Compare before/after statistics
- Conduct spot checks and sample reviews

5. Documentation:

- Record all cleaning decisions and rationale
- Create data dictionaries
- Document assumptions and limitations

2.4.3 Tools and Techniques in R

The are several R packages and functions designed to facilitate data cleaning tasks:

- `tidyverse` (dplyr, tidyr): Data manipulation and reshaping
- `janitor`: Data cleaning utilities and tabulation
- `naniar`, `mice`: Missing data visualization and imputation
- `stringr`, `stringi`: Text cleaning and manipulation
- `lubridate`: Date-time parsing and manipulation
- `validate`, `assertr`: Data validation
- `dedupr`, `RecordLinkage`: Deduplication

Business example: A retail company discovers that their customer database contains 15% missing email addresses, duplicate customer records with slight name variations (e.g., "John Smith" vs. "J. Smith"), inconsistent country codes ("USA" vs. "US" vs. "United States"), and impossible birth dates (e.g., year 1899). The data cleaning process involves:

1. Flagging missing emails for targeted collection campaigns
2. Using fuzzy matching to merge duplicate customer profiles while preserving transaction history
3. Standardizing country codes using ISO standards
4. Validating and correcting birth dates using business rules (customers must be 18+ years old)
5. Creating a cleaned, analysis-ready dataset that improves customer segmentation accuracy and marketing campaign targeting.

2.5 Association: Measuring Relationships Between Variables

Association analysis examines how variables relate to each other, quantifying the strength, direction, and nature of relationships. Understanding associations is fundamental for identifying patterns, making predictions, and uncovering causal mechanisms in business analytics.

2.5.1 Correlation Analysis

Correlation measures the strength and direction of linear relationships between two variables, producing coefficients ranging from -1 (perfect negative correlation) to +1 (perfect positive correlation), with 0 indicating no linear relationship.

Types of Correlation Coefficients:

Pearson Correlation (r):

- Measures linear relationships between continuous variables
- Assumes normally distributed data
- Sensitive to outliers
- Interpretation: r = 0.7-1.0 (strong), 0.4-0.7 (moderate), 0.1-0.4 (weak)

Spearman Rank Correlation ( ):

- Measures monotonic relationships (not necessarily linear)
- Non-parametric alternative to Pearson
- Works with ordinal data
- Robust to outliers and non-normal distributions

Kendall's Tau ( ):

- Measures ordinal associations
- Better for small sample sizes
- More robust than Spearman for tied ranks

Key Considerations:

- **Correlation Causation**: Strong correlation doesn't imply one variable causes changes in another
- **Spurious correlations**: Unrelated variables may correlate due to confounding factors or coincidence
- **Non-linear relationships**: Correlation coefficients may miss curved or complex patterns
- **Range restrictions**: Limited variable ranges can artificially reduce correlation

Business example: Analyzing the correlation between advertising spend and sales revenue (r = 0.82) suggests a strong positive linear relationship, but further analysis is needed to establish causality and account for seasonal effects, competitor actions, and other factors.

```
# Loading necessary libraries
library(ourdata)
library(dplyr)

# Getting the current working directory
getwd()
```

```
[1] "/home/runner/work/DrBenjamin.github.io/DrBenjamin.github.io/source-
analytical"
```

```
# Creating data frames from the ourdata package datasets
hdi_df <- hdi
imr_df <- imr

# Combining two data frames
combined_df <- ourdata::combine(imr_df$name, hdi_df$country, imr_df$`deaths/1`, hdi_df$Human
                                col1 = "Country", col2 = "IMR", col3 = "HDI")
```

```
'data.frame':   181 obs. of  3 variables:
 $ Country: chr  "Afghanistan" "Somalia" "Central African Republic" "Equatorial Guinea" ...
 $ IMR    : num  101.3 83.6 80.5 77.4 71.2 ...
 $ HDI    : num  0.496 0.404 0.414 0.674 0.467 0.419 0.416 0.388 0.493 0.419 ...
```

```
# or combining with dplyr
combined_df <- inner_join(imr_df, hdi_df, by = c("name" = "country")) %>%
  select(Country = name, IMR = `deaths/1`, HDI = HumanDevelopmentIndex_2024)

# Calculating Pearson correlation coefficient
pearson_cor <- cor(combined_df$HDI, combined_df$IMR, method = "pearson")

# Calculating Spearman correlation coefficient (rank-based, robust to outliers)
spearman_cor <- cor(combined_df$HDI, combined_df$IMR, method = "spearman")
```

```r
# Performing correlation test (includes p-value and confidence interval)
cor_test <- cor.test(combined_df$HDI, combined_df$IMR, method = "pearson")

# Printing correlation results
cat("Correlation Analysis: HDI vs IMR\n")
```

```
Correlation Analysis: HDI vs IMR
```

```r
cat("================================\n\n")
```

```
================================
```

```r
cat("Pearson Correlation Coefficient:", round(pearson_cor, 4), "\n")
```

```
Pearson Correlation Coefficient: -0.8588
```

```r
cat("Spearman Correlation Coefficient:", round(spearman_cor, 4), "\n\n")
```

```
Spearman Correlation Coefficient: -0.9172
```

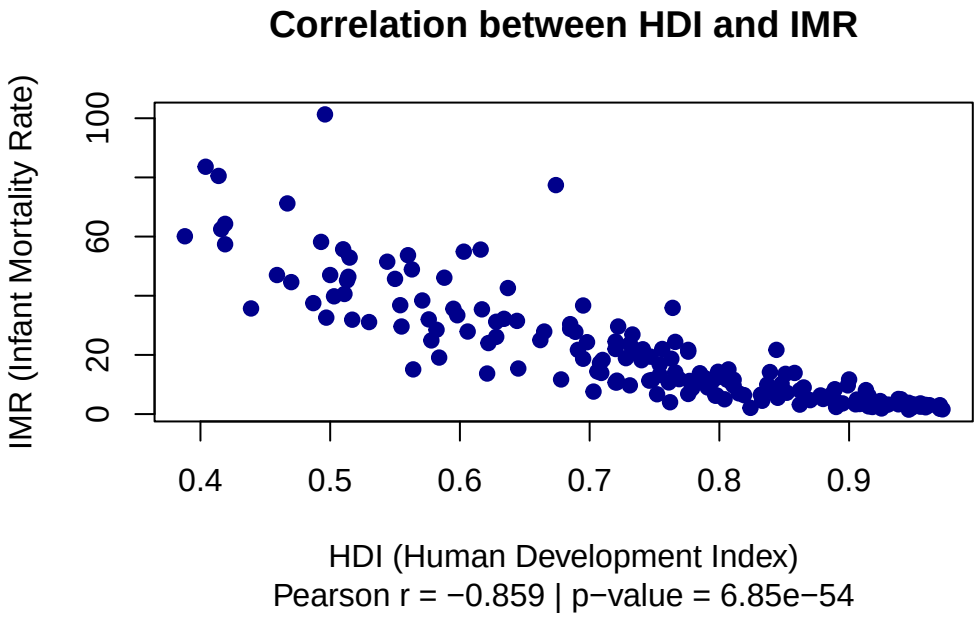
```r
cat("Correlation Test Results:\n")
```

```
Correlation Test Results:
```

```r
print(cor_test)
```

```

    Pearson's product-moment correlation

data:  combined_df$HDI and combined_df$IMR
t = -22.432, df = 179, p-value < 2.2e-16
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 -0.8928546 -0.8150919
sample estimates:
       cor
-0.8588439
```

```r
# Creating scatter plot with correlation annotation
plot(combined_df$HDI, combined_df$IMR,
     main = "Correlation between HDI and IMR",
     sub = paste("Pearson r =", round(pearson_cor, 3),
                 "| p-value =", format(cor_test$p.value, scientific = TRUE, digits = 3)),
     ylab = "IMR (Infant Mortality Rate)",
     xlab = "HDI (Human Development Index)",
     pch = 19,
     col = "darkblue")
```




```
# Adding correlation interpretation text
if (abs(pearson_cor) >= 0.7) {
  strength <- "strong"
} else if (abs(pearson_cor) >= 0.4) {
  strength <- "moderate"
} else {
  strength <- "weak"
}
direction <- ifelse(pearson_cor < 0, "negative", "positive")
cat("\nInterpretation:\n")
```

```
Interpretation:
```

```
cat("There is a", strength, direction, "correlation between HDI and IMR.\n")
```

```
There is a strong negative correlation between HDI and IMR.
```

2.5.2 Regression Analysis

Regression models relationships between variables to predict outcomes, estimate effects, and understand dependencies. It quantifies how changes in independent variables (predictors) relate to changes in dependent variables (outcomes).

Simple Linear Regression:

- Models relationship between one predictor and one outcome
- Equation: Y = + X +
- (intercept): predicted Y when X = 0
- (slope): change in Y for one-unit increase in X

- (error term): unexplained variation

Multiple Linear Regression:

- Models relationships with multiple predictors
- Equation: Y = + X + X + … + X +
- Controls for confounding variables
- Allows estimation of partial effects

Model Evaluation Metrics:

R-squared (R^2):

- Proportion of variance explained by the model (0 to 1)
- Higher values indicate better fit
- Can be inflated by adding predictors

Adjusted R-squared:

- Penalizes model complexity
- Preferred for comparing models with different numbers of predictors

Root Mean Squared Error (RMSE):

- Average prediction error in original units
- Lower values indicate better predictions

Residual Analysis:

- Examining prediction errors to validate assumptions
- Checking for patterns, heteroscedasticity, and normality

Key Assumptions:

- **Linearity**: Relationship between variables is linear
- **Independence**: Observations are independent
- **Homoscedasticity**: Constant variance of errors
- **Normality**: Residuals are normally distributed
- **No multicollinearity**: Predictors are not highly correlated (for multiple regression)

Types of Relationships:

Positive Association:

- Both variables increase together
- Example: Education level and income

Negative Association:

- One variable increases as the other decreases
- Example: Price and demand

No Association:

- Variables are unrelated
- Changes in one don't predict changes in the other

Non-linear Association:

- Curved or complex relationships
- May require polynomial regression or transformation

Practical Applications in Business:

Predictive Modeling:

- Sales forecasting based on historical data
- Customer lifetime value prediction
- Demand estimation

Risk Assessment:

- Credit scoring models
- Insurance premium calculation
- Investment risk evaluation

Optimization:

- Pricing strategies
- Resource allocation
- Marketing mix optimization

Causal Inference:

- Treatment effect estimation
- A/B testing analysis
- Policy impact evaluation

Business example: A subscription-based software company builds a multiple regression model to predict customer churn:

```
Churn Probability =   +  (Login_Frequency) +  (Support_Tickets) +  (Feature_Usage) +  (Cont
```

Results show:

- Login frequency has a strong negative effect (= -0.35): more active users are less likely to churn
- Support tickets have a positive effect (= 0.18): customers with issues are more likely to leave
- Feature usage has a moderate negative effect (= -0.22)
- Contract length has a strong negative effect (= -0.41): longer commitments reduce churn

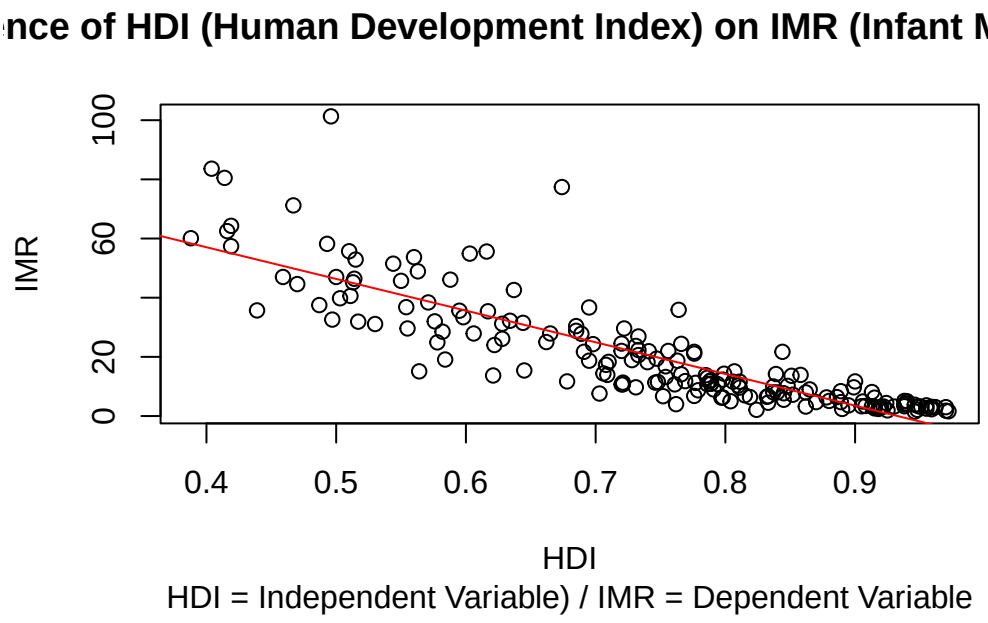
The model achieves $R^2 = 0.67$, explaining 67% of churn variance. These insights inform retention strategies: improving onboarding to increase feature adoption, proactively addressing support issues, and incentivizing longer contracts.

Example linear regression analysis and storytelling:

```r
# Loading necessary libraries
library(ourdata)
library(dplyr)

# Creating data frames from the ourdata package datasets
hdi_df <- hdi
imr_df <- imr

# Combining two data frames
combined_df <- ourdata::combine(imr_df$name, hdi_df$country, imr_df$`deaths/1`, hdi_df$Human
                                col1 = "Country", col2 = "IMR", col3 = "HDI")
```

```
'data.frame':   181 obs. of  3 variables:
 $ Country: chr  "Afghanistan" "Somalia" "Central African Republic" "Equatorial Guinea" ...
 $ IMR    : num  101.3 83.6 80.5 77.4 71.2 ...
 $ HDI    : num  0.496 0.404 0.414 0.674 0.467 0.419 0.416 0.388 0.493 0.419 ...
```

```r
# or combining with dplyr
combined_df <- inner_join(imr_df, hdi_df, by = c("name" = "country")) %>%
  select(Country = name, IMR = `deaths/1`, HDI = HumanDevelopmentIndex_2024)

# Creating scatter plot
plot(combined_df$HDI, combined_df$IMR,
  main = "Influence of HDI (Human Development Index) on IMR (Infant Mortality Rate)",
  sub = "HDI = Independent Variable) / IMR = Dependent Variable",
  ylab = "IMR", xlab = "HDI")

# Adding regression line
model <- lm(combined_df$IMR ~ combined_df$HDI)
abline(model, col = "red")
```



```r
# Showing summary of the model
summary(model)
```

```
Call:
lm(formula = combined_df$IMR ~ combined_df$HDI)

Residuals:
    Min      1Q  Median      3Q     Max
-24.387  -5.474  -0.017   4.470  54.530

Coefficients:
                Estimate Std. Error t value Pr(>|t|)
(Intercept)       99.893      3.623   27.57   <2e-16 ***
combined_df$HDI -107.103      4.775  -22.43   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 9.723 on 179 degrees of freedom
Multiple R-squared:  0.7376,    Adjusted R-squared:  0.7361
F-statistic: 503.2 on 1 and 179 DF,  p-value: < 2.2e-16
```

Conclusion and interpretation of the results:

The fitted linear model (IMR ~ HDI) shows a very strong, negative association: the HDI coefficient is −113.23 (SE = 4.73, t = −23.92, p < 2e-16).

This means that a one-unit increase in HDI (note: HDI ranges roughly 0–1) is associated with a predicted decrease in infant mortality of about 113 deaths per 1,000 live births.

Practical meaning Higher human development (better education, income and health components of HDI) is strongly associated with lower infant mortality.

Caveats and diagnostics to check Correlation causation: this is observational. Confounding (e.g., health spending, access to care, urbanization) could drive both HDI and IMR. Avoid causal claims without further analysis.

Summary HDI and IMR are strongly negatively associated: higher HDI is linked to substantially lower infant mortality (HDI explains ~77% of IMR variation here), but confirmatory causal analysis and diagnostic checks are needed before policy attribution.

2.5.3 Correlation vs. Regression

Correlation:

- Symmetric relationship (X-Y same as Y-X)
- No distinction between dependent and independent variables
- Describes strength and direction only
- Single coefficient summarizes relationship

Regression:

- Asymmetric relationship (predicting Y from X)

- Clear dependent and independent variables
- Provides prediction equation
- Estimates effect sizes and intercepts
- Allows statistical inference about relationships

Both techniques complement each other in comprehensive data analysis, with correlation providing initial relationship exploration and regression enabling detailed modeling and prediction.

3 Inferential statistics

Inferential statistics uses sample data to make generalizations about a larger population, quantifying uncertainty and supporting evidence-based decision-making in business contexts. It encompasses hypothesis testing, estimation, and prediction, forming the foundation for advanced analytics.

3.1 Basic concepts of statistical inference

Statistical inference involves drawing conclusions about a population based on sample data, quantifying uncertainty through probability theory. Key concepts include:

- **Population vs. Sample**: The population is the entire group of interest, while a sample is a subset used for analysis. In business, samples are often drawn from customer bases, sales data, or market segments to infer broader trends.
- **Parameters vs. Statistics**: Parameters are numerical characteristics of populations (e.g., population mean), while statistics are calculated from samples (e.g., sample mean) and used to estimate parameters.
- **Sampling Distribution**: The probability distribution of a statistic (e.g., sample mean) across all possible samples from the population. It forms the basis for estimating variability and constructing confidence intervals.

3.2 Quantification of probability through random variables

Random variables are mathematical representations of uncertain outcomes in business contexts, allowing quantification of probabilities associated with different events. They can be classified as:

- **Discrete Random Variables**: Take on countable values (e.g., number of customer purchases, number of defective products). Probability mass functions (PMFs) describe the probabilities of each outcome.
- **Continuous Random Variables**: Take on an infinite number of values within a range (e.g., customer wait times, sales amounts). Probability density functions (PDFs) describe the likelihood of outcomes within intervals.

3.3 Hypothesis testing

Hypothesis testing is a fundamental statistical method used to make inferences about population parameters based on sample data (Illowsky & Dean, 2018). It provides a systematic framework for evaluating claims about populations using sample evidence, enabling data-driven decision making in business contexts.

3.3.1 Core Concepts

A statistical hypothesis test involves formulating two competing hypotheses:

- **Null hypothesis (H)**: The status quo or default position, typically representing no effect or no difference

- **Alternative hypothesis (H or H)**: The research hypothesis representing the effect or difference we seek to detect

The process involves calculating a test statistic from sample data and comparing it to a critical value or determining a p-value to make decisions about rejecting or failing to reject the null hypothesis (GeeksforGeeks, 2024).

3.3.2 Key Components

Test Statistics: Standardized measures that quantify how far sample data deviates from what would be expected under the null hypothesis.

Significance Level ( ): The probability threshold for rejecting the null hypothesis, commonly set at 0.05 (5%).

P-value: The probability of observing test results at least as extreme as those obtained, assuming the null hypothesis is true.

Critical Region: The range of values for which the null hypothesis is rejected.

3.3.3 Types of Errors

- **Type I Error ( )**: Rejecting a true null hypothesis (false positive)
- **Type II Error ( )**: Failing to reject a false null hypothesis (false negative)
- **Statistical Power (1-)**: The probability of correctly rejecting a false null hypothesis

3.3.4 Reference Materials

For comprehensive coverage of hypothesis testing concepts, methodologies, and applications, consult:

- **Theoretical Foundation**: Introductory Statistics provides detailed explanations of hypothesis testing principles and procedures (Illowsky & Dean, 2018)
- **Visual Guide**: Hypothesis Testing Overview offers a visual representation of key concepts
- **Detailed Methodology**: Hypothesis Testing Documentation contains comprehensive methodological information
- **Online Resource**: Additional perspectives on hypothesis testing applications can be found in the GeeksforGeeks guide (GeeksforGeeks, 2024)

Understanding hypothesis testing is essential for making informed business decisions based on data analysis, forming the foundation for advanced statistical inference and predictive analytics in business contexts.

3.4 Confidence Intervals, P-Values, and Statistical Tests

Confidence intervals, p-values, and statistical tests are interconnected concepts in inferential statistics that work together to support evidence-based decision-making and quantify uncertainty in business analytics.

3.4.1 Confidence Intervals

Confidence intervals (CIs) provide a range of plausible values for population parameters based on sample data, expressing uncertainty around point estimates with a specified confidence level (typically 95%).

Interpretation:

- A 95% CI means that if we repeatedly sampled from the population and calculated confidence intervals, approximately 95% of those intervals would contain the true population parameter
- Wider intervals indicate greater uncertainty; narrower intervals suggest more precise estimates
- The interval provides both a point estimate (usually the center) and a margin of error

Common Types:

- **CI for means**: Estimates average values (e.g., average customer satisfaction score: 7.2 ± 0.5)
- **CI for proportions**: Estimates percentages (e.g., conversion rate: 12% ± 2%)
- **CI for differences**: Compares groups (e.g., A/B test difference: 3.5% ± 1.2%)
- **CI for regression coefficients**: Quantifies predictor effects with uncertainty

Business application: A marketing team tests a new email campaign and observes a 15% click-through rate in a sample of 500 customers. The 95% CI is [12.3%, 17.7%], indicating they can be 95% confident the true population click-through rate lies within this range.

3.4.2 P-Values

P-values quantify the strength of evidence against the null hypothesis, representing the probability of observing results as extreme as or more extreme than those obtained, assuming the null hypothesis is true.

Interpretation Guidelines:

- $\mathbf{p < 0.01}$: Very strong evidence against H (highly significant)
- $\mathbf{p < 0.05}$: Strong evidence against H (statistically significant)
- $\mathbf{p < 0.10}$: Moderate evidence against H (marginally significant)
- $\mathbf{p \quad 0.10}$: Insufficient evidence to reject H (not significant)

Key Considerations:

- P-values do NOT measure the probability that the null hypothesis is true
- P-values do NOT measure effect size or practical importance
- Smaller p-values indicate stronger evidence, but don't guarantee meaningful effects
- Statistical significance (small p-value) practical significance (meaningful effect)
- P-values can be influenced by sample size: large samples may yield significant p-values for trivial effects

Common Misinterpretations to Avoid:

- "p = 0.03 means there's a 3% chance the null hypothesis is true"
- "p > 0.05 proves the null hypothesis is correct"

- "p = 0.03 means that if the null hypothesis were true, we'd see results this extreme only 3% of the time"

Business application: Testing whether a new product feature increases user engagement, researchers obtain p = 0.02, indicating that if the feature had no effect, results this extreme would occur only 2% of the time—strong evidence the feature has an impact.

3.4.3 Common Statistical Tests

Statistical tests evaluate hypotheses about population parameters, each suited for specific data types and research questions.

Tests for Means

t-test:

- **One-sample t-test**: Compare sample mean to known value (e.g., Is average satisfaction score different from 7.0?)
- **Independent samples t-test**: Compare means between two groups (e.g., Do customers in Region A spend more than Region B?)
- **Paired samples t-test**: Compare means for related observations (e.g., Before/after treatment comparisons)

ANOVA (Analysis of Variance):

- Compares means across three or more groups
- Identifies whether at least one group differs significantly
- Example: Comparing sales performance across four sales regions

Tests for Proportions

Z-test for proportions:

- Compare sample proportion to known value
- Compare proportions between two groups
- Example: Is conversion rate significantly higher for Treatment A vs. Control?

Chi-square test:

- Tests independence between categorical variables
- Example: Is customer satisfaction level related to product category?

Tests for Associations

Correlation test:

- Evaluates significance of correlation coefficients
- Tests whether observed correlation differs from zero
- Example: Is there a significant relationship between advertising spend and revenue?

Regression F-test:

- Tests overall model significance
- Evaluates whether predictors collectively explain variance
- Example: Do customer demographics significantly predict purchase behavior?

Non-parametric Tests:

When data violates normality assumptions, use distribution-free alternatives: - **Mann-Whitney U test**: Non-parametric alternative to independent t-test - **Wilcoxon signed-rank test**: Non-parametric alternative to paired t-test - **Kruskal-Wallis test**: Non-parametric alternative to ANOVA

3.4.4 Interconnections

These concepts work together in hypothesis testing:

Relationship between CIs and p-values:

- If a 95% CI for a difference excludes zero, the corresponding p-value will be < 0.05
- If a 95% CI includes zero, the p-value will be > 0.05
- CIs provide more information than p-values: they show effect size, direction, and precision

Relationship between p-values and significance level ( ):

- If p < (commonly 0.05), reject the null hypothesis
- If p , fail to reject the null hypothesis
- The level defines Type I error rate (false positive rate)

Choosing the Right Test:

1. **Data type**: Continuous, categorical, ordinal?
2. **Number of groups**: One, two, or more?
3. **Independence**: Related or unrelated samples?
4. **Assumptions**: Normality, equal variances?
5. **Research question**: Difference, association, or prediction?

Business Decision Framework:

1. **Statistical Significance**: Does the p-value indicate a real effect? ($p < 0.05$)
2. **Effect Size**: Is the magnitude practically meaningful? (examine CIs and Cohen's d)
3. **Business Impact**: Does this translate to meaningful outcomes? (revenue, retention, satisfaction)
4. **Cost-Benefit**: Do benefits outweigh implementation costs?

Example Integration: An e-commerce company tests a new checkout design:

- **Sample data**: 1,000 users per version (old vs. new)
- **Conversion rates**: Old = 12.0%, New = 14.5%
- **Statistical test**: Two-proportion z-test
- **Results**:
  - Difference: 2.5 percentage points
  - 95% CI: [0.8%, 4.2%]
  - p-value: 0.004
- **Interpretation**:
  - **Statistically significant**: $p = 0.004 < 0.05$ (strong evidence of a difference)
  - **Effect size**: 2.5 percentage point increase (20.8% relative improvement)

  - **Precision**: 95% confident the true improvement is between 0.8% and 4.2%

- **Business decision**: Implement new design; with 100,000 monthly users, this represents an estimated 2,500 additional conversions monthly

This integrated approach ensures decisions are statistically sound, practically meaningful, and aligned with business objectives.

3.5 Inferential statistics

in the programming language R, translating theoretical knowledge into practical applications.

4 Predictive analytics

Predictive analytics uses statistical techniques and machine learning algorithms to analyze historical data and make predictions about future events or behaviors. It enables businesses to anticipate trends, optimize operations, and enhance decision-making.

4.1 Data mining techniques

These are methods used to extract patterns and insights from large datasets.

4.2 Regression analysis

Regression analysis is a statistical technique used to model and analyze the relationships between a dependent variable and one or more independent variables. It helps in understanding how changes in the independent variables impact the dependent variable, enabling businesses to make informed predictions and decisions.

4.3 Forecasting in predicting future business outcomes

Forecasting involves using historical data and statistical models to make informed predictions about future business outcomes. It helps organizations anticipate trends, allocate resources, and plan strategically.

5 Literature

All references for this course.

5.1 Essential Readings

Bruce, P. and A. Bruce (2020). *Practical Statistics for Data Scientists, 2nd Edition*. https://learning.oreilly.com/library/view/practical-statistics-for/9781492072935/preface01.html.

Çetinkaya-Rundel, M. and J. Hardin (2021). *Introduction to Modern Statistics*. https://www.openintro.org/book/ims/. https://github.com/DrBenjamin/Analytical-Skills-for-Business/blob/491a9a84dd0227aab44e0a6db7e6330830a05a6b/literature/Introduction_to_Modern_Statistics_2e.pdf/?raw=true.

Stephenson, P. (2023). *Data Science Practice*. https://datasciencepractice.study/.

5.2 Further Readings

Békés, G. and G. Kézdi (2021). *Resources for Data Analysis for Business, Economics, and Policy*. Instructor resources: https://www-cambridge-org.eux.idm.oclc.org/highereducation/books/data-analysis-for-business-economics-and-policy/D67A1B0B56176D6D6A92E27F3F82AA20/. https://github.com/DrBenjamin/Analytical-Skills-for-Business/blob/c2ec1b2061c7dc36200977cfd58daf6020c1c774/literature/B%C3%A9k%C3%A9s_Data%20Analysis%20for%20Business%2C%20Economics%2C%20and%20Policy_2021_First%20Day%20of%20Class%20Slides.pdf/?raw=true.}

Britannica (2025). *Computer science | Definition, Types, & Facts | Britannica*. https://www.britannica.com/science/computer-science.

Dougherty, J. and I. Ilyankou (2025). *Hands-On Data Visualization*. https://handsondataviz.org/.

Evans, J. R. (2020). "Business Analytics' '.

GeeksforGeeks (2024a). *Understanding Hypothesis Testing*. Online resource for software testing and statistical concepts. https://www.geeksforgeeks.org/software-testing/understanding-hypothesis-testing/.

GeeksforGeeks, G. (2024b). *Difference between Git and GitHub*. Comprehensive comparison of Git and GitHub for version control. https://www.geeksforgeeks.org/git/difference-between-git-and-github/.

GitHub (2024). *About GitHub and Git*. Official GitHub documentation on Git and GitHub fundamentals. https://docs.github.com/en/get-started/start-your-journey/about-github-and-git.

Illowsky, B. and S. L. Dean (2018). *Introductory Statistics*. OpenStax, Rice University, p. 905. ISBN: 1938168208. https://github.com/DrBenjamin/Analytical-Skills-for-Business/blob/c2ec1b2061c7dc36200977cfd58daf6020c1c774/literature/Introductory%20Statistics.pdf/?raw=true.}

Irizarry, R. A. (2024). "Advanced Data Science: Statistics and Prediction Algorithms Through Case Studies' '.

http://rafalab.dfci.harvard.edu/dsbook-part-2.

Kumar, U. D. (2017). "Business Analytics: The Science of Data-Driven Decision Making' '.

Pochiraju, B. and S. Seshadri, ed. (2019). *Essentials of Business Analytics*. Vol. 264. https://link.springer.com/10.1007/978-3-319-68837-4. Cham: Springer International Publishing. ISBN: 978-3-319-68836-7. DOI: 10.1007/978-3-319-68837-4. https://github.com/DrBenjamin/Analytical-Skills-for-Business/blob/c2ec1b2061c7dc36200977cfd58daf6020c1c774/literature/Essentials%20of%20Business%20Analytics.pdf/?raw=true.}

Vaughan, D. (2020). "Analytical Skills for AI and Data Science' '.

https://learning.oreilly.com/library/view/analytical-skills-for/9781492060932/preface01.html#idm46388898852872.

6 Example Exam

To prepare for the exam, please review the following example exam with solutions:

Example Exam